

1. Introduction

The in-memory solution for the Oasis Kernel is called the kernel tools or "ktools". ktools is an independent "specification" of a set of processes which means that it defines the processing architecture and data structures. The framework is implemented as a set of components called the "reference model" which can then be adapted for particular model or business needs.

The code can be compiled in Linux, POSIX-compliant Windows and native Windows. The installation instructions can be found in [README.html](#).

Background

The Kernel performs the core Oasis calculations of computing effective damageability distributions, Monte-Carlo sampling of ground up loss, the financial module calculations, which apply insurance policy terms and conditions to the sampled losses, and finally some common catastrophe model outputs.

The early releases of Oasis used a SQL-compliant database to perform all calculations. Release 1.3 included the first "in-memory" version of the Oasis Kernel written in C++ and C to provide streamed calculation at high computational performance, as an alternative to the database calculation. The scope of the in-memory calculation was for the most intensive Kernel calculations of ground up loss sampling and the financial module. This in-memory variant was first delivered as a stand-alone toolkit "ktools" with R1.4.

With R1.5, a Linux-based in-memory calculation back-end was released, using the reference model components of ktools. The range of functionality of ktools was still limited to ground up loss sampling, the financial module and single output workflows, with effective damage distributions and output calculations still being performed in a SQL-compliant database.

In 2016 the functionality of ktools was extended to include the full range of Kernel calculations, including effective damageability distribution calculations and a wider range of financial module and output calculations. The data stream structures and input data formats were also substantially revised to handle multi-peril models, user-definable summary levels for outputs, and multiple output workflows.

In 2018 the Financial Module was extended to perform net loss calculations for per occurrence forms of reinsurance, including facultative reinsurance, quota share, surplus share, per risk and catastrophe excess of loss treaties.

Architecture

The Kernel is provided as a toolkit of components ("ktools") which can be invoked at the command line. Each component is a separately compiled executable with a binary data stream of inputs and/or outputs.

The principle is to stream data through the calculations in memory, starting with generating the damage distributions and ending with calculating the user's required result, before writing the output to disk. This is done on an event-by-event basis, which means at any one time the compute server only has to hold the model data for a single event in its memory, per process. The user can run the calculation across multiple processes in parallel, specifying the analysis workflow and number of processes in a script file appropriate to the operating system.

Language

The components can be written in any language as long as the data structures of the binary streams are adhered to. The current set of components have been written in POSIX-compliant C++. This means that they can be compiled in Linux and Windows using the latest GNU compiler toolchain.

Components

The components in the Reference Model can be summarized as follows;

- **Core components** perform the core kernel calculations and are used in the main analysis workflow.
- **Output components** perform specific output calculations at the end of the analysis workflow. Essentially, they produce various statistical summaries of the sampled losses from the core calculation.

- **Data conversion components** are provided to enable users to convert model and exposure data into the required binary formats.
- **Stream conversion components** are provided to inspect the data streams flowing between the core components, for more detailed analysis or debugging purposes.

Usage

Standard piping syntax can be used to invoke the components at the command line. It is the same syntax in Windows DOS, Linux terminal or Cygwin (a Linux emulator for Windows). For example the following command invokes eve, getmodel, gulcalc, fmcalc, summarycalc and eltcac, and exports an event loss table output to a csv file.

```
$ eve 1 1 | getmodel | gulcalc -r -S100 -a1 -i - | fmcalc | summarycalc -f -1 - | eltcac > elt.csv
```

Example python scripts are provided along with a binary data package in the /examples folder to demonstrate usage of the toolkit. For more guidance on how to use the toolkit, see [Workflows](#).

[Go to 2. Data streaming architecture overview](#)

[Back to Contents](#)

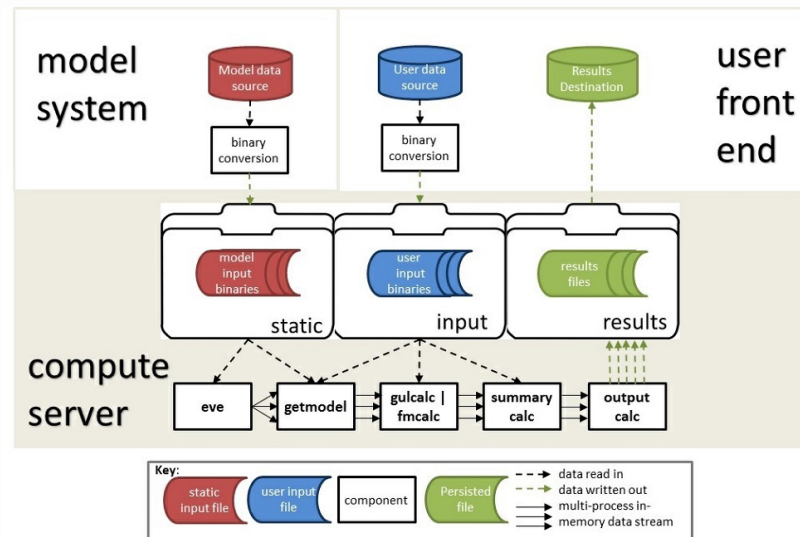
Table of Contents

- 1. Introduction
- 2. Data streaming architecture overview
- 3. Specification
- 4. Reference model
 - 4.1 Core components
 - 4.2 Output components
 - 4.3 ORD output components
 - 4.4 Data conversion components
 - 4.5 Stream conversion components
 - 4.6 Validation components
- 5. Financial Module
- 6. Workflows
- Appendix A Random numbers
- Appendix B FM profiles
- Appendix C Multi-peril model support

2. Data Streaming Framework Overview

This is the general data streaming framework showing the core components of the toolkit.

Figure 1. Data streaming framework



The architecture consists of;

- A **model system** which provides the model data in the correct binary format. This is a one time set-up task.
- A **user front end**, which provides the users data for an analysis against the model.
- A **compute server**, where the computations are performed. The required model data is held in a 'static' folder, and the required user data is held in an 'input' folder.

The conversion of input data to binary format is shown in the diagram as occurring outside of the compute server, but this could be performed within the compute server. ktools provides a full set of binary conversion tools from csv input files which can be deployed elsewhere.

The in-memory data streams are initiated by the process 'eve' (meaning 'event emitter') and shown by solid arrows. The read/write data flows are shown as dashed arrows.

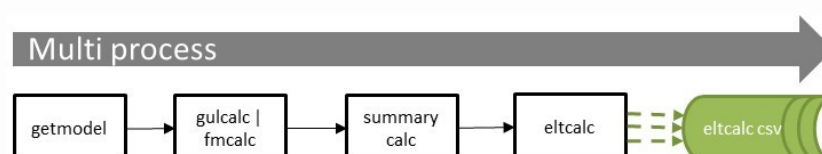
The calculation components are *getmodel*, *gulcalc*, *fmcalc*, *summarycalc* and *outputcalc*. The streamed data passes through the components in memory one event at a time and are written out to a results file on the compute server. The user can then retrieve the results (csvs) and consume them in their BI system.

The reference model demonstrates an implementation of the core calculation components, along with the data conversion components which convert binary files to csv files.

The analysis workflows are controlled by the user, not the toolkit, and they can be as simple or as complex as required.

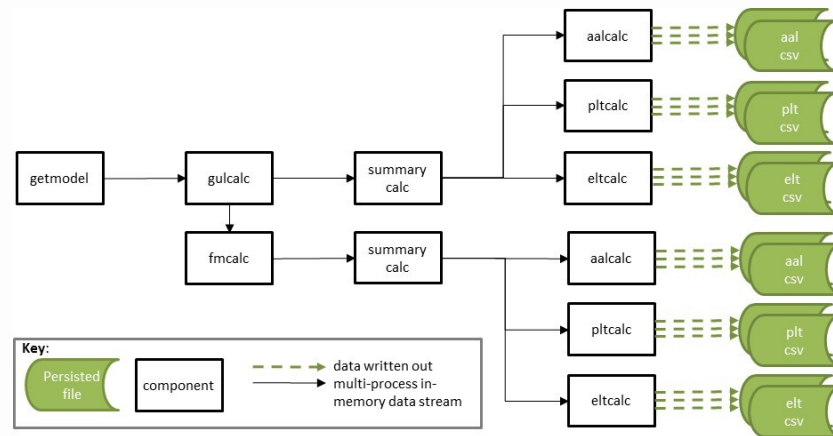
The simplest workflow is single or parallel processing to produce a single result. This minimises the amount of disk I/O at each stage in the calculation, which performs better than saving intermediate results to disk. This workflow is shown in Figure 2.

Figure 2. Single output processing



However it is possible to stream data from one process into to several processes, allowing the calculation of multiple outputs simultaneously, as shown in Figure 3.

Figure 3. Multiple output processing



For multi-output, multi-process workflows, Linux operating systems provide 'named pipes' which in-memory data streams can be diverted to and manipulated as if they were files, and 'tee' which sends a stream from one process into multiple processes. This means the core calculation is not repeated for each output, as it would be if several single-output workflows were run.

[Go to 3. Specification](#)

[Back to Contents](#)

3. Specification

Introduction

This section specifies the data stream structures and core components in the in-memory kernel.

The data stream structures are;

- **cdf stream**
- gulcalc stream(deprecated)
- **loss stream**
- **summary stream**

The stream data structures have been designed to minimise the volume flowing through the pipeline, using data packet 'headers' to remove redundant data. For example, indexes which are common to a block of data are defined as a header record and then only the variable data records that are relevant to the header key are part of the data stream. The names of the data fields given below are unimportant, only their position in the data stream is important in order to perform the calculations defined in the program.

The components are;

- **eve**
- **getmodel**
- **gulcalc**
- **fmcalc**
- **summarycalc**
- **outputcalc**

The components have a standard input (stdin) and/or output (stdout) data stream structure. eve is the stream-initiating component which only has a standard output stream, whereas "outputcalc" (a generic name representing an extendible family of output calculation components) is a stream-terminating component with only a standard input stream.

An implementation of each of the above components is provided in the [Reference Model](#), where usage instructions and command line parameters are provided. A functional overview is given below.

Stream types

The architecture supports multiple stream types. Therefore a developer can define a new type of data stream within the framework by specifying a unique stream_id of the stdout of one or more of the components, or even write a new component which performs an intermediate calculation between the existing components.

The stream_id is the first 4 byte header of the stdout streams. The higher byte is reserved to identify the type of stream, and the 2nd to 4th bytes hold the identifier of the stream. This is used for validation of pipeline commands to report errors if the components are not being used in the correct order.

The current reserved values are as follows;

Higher byte;

Byte 1	Stream name
0	cdf
1	gulcalc (deprecated)
2	loss
3	summary

Reserved stream_ids;

Byte 1	Bytes 2-4	Description
0	1	cdf - Oasis format effective damageability CDF output
1	1	gulcalc - Oasis format item level ground up loss sample output (deprecated)
1	2	gulcalc - Oasis format coverage level ground up loss sample output (deprecated)
2	1	loss - Oasis format loss sample output (any loss perspective)
3	1	summary - Oasis format summary level loss sample output

The supported standard input and output streams of the reference model components are summarized here;

Component	Standard input	Standard output	Stream option parameters
getmodel	none	0/1 cdf	none
gulcalc	0/1 cdf	2/1 loss	-i -a{}
fmcalc	2/1 loss	2/1 loss	none
summarycalc	2/1 loss	3/1 summary	-i input from gulcalc, -f input from fmcalc
outputcalc	3/1 summary	none	none

Stream structure

cdf stream

Stream header packet structure

Name	Type	Bytes	Description	Example
stream_id	int	1/3	Identifier of the data stream type.	0/1

Data header packet structure

Name	Type	Bytes	Description	Example
event_id	int	4	Oasis event_id	4545
areaperil_id	int	4	Oasis areaperil_id	345456
vulnerability_id	int	4	Oasis vulnerability_id	345
no_of_bins	int	4	Number of records (bins) in the data package	20

Data packet structure (record repeated no_of_bin times)

Name	Type	Bytes	Description	Example
prob_to	float	4	The cumulative probability at the upper damage bin threshold	0.765
bin_mean	float	4	The conditional mean of the damage bin	0.45

[Return to top](#)

loss stream

Stream header packet structure

Name	Type	Bytes	Description	Example
stream_id	int	1/3	Identifier of the data stream type.	2/1
no_of_samples	int	4	Number of samples	100

Data header packet structure

Name	Type	Bytes	Description	Example
event_id	int	4	Oasis event_id	4545
item_id /output_id	int	4	Oasis item_id (gulcalc) or output_id (fmcalc)	300

Data packet structure

Name	Type	Bytes	Description	Example
sidx	int	4	Sample index	10
loss	float	4	The loss for the sample	5625.675

The data packet may be a variable length and so a `sidx` of 0 identifies the end of the data packet.

There are five values of `sidx` with special meaning as follows;

sidx	Meaning	Required / optional
-5	maximum loss	optional
-4	chance of loss	optional
-3	impacted exposure	required
-2	numerical integration standard deviation loss	optional
-1	numerical integration mean loss	required

`sidx` -5 to -1 must come at the beginning of the data packet before the other samples in ascending order (-5 to -1).

[Return to top](#)

summary stream

Stream header packet structure

Name	Type	Bytes	Description	Example
<code>stream_id</code>	int	1/3	Identifier of the data stream type.	3/1
<code>no_of_samples</code>	int	4	Number of samples	100
<code>summary_set</code>	int	4	Identifier of the summary set	2

Data header packet structure

Name	Type	Bytes	Description	Example
<code>event_id</code>	int	4	Oasis <code>event_id</code>	4545
<code>summary_id</code>	int	4	Oasis <code>summary_id</code>	300
<code>exposure_value</code>	float	4	Impacted exposure (sum of <code>sidx</code> -3 losses for <code>summary_id</code>)	987878

Data packet structure

Name	Type	Bytes	Description	Example
<code>sidx</code>	int	4	Sample index	10
<code>loss</code>	float	4	The loss for the sample	5625.675

The data packet may be a variable length and so a `sidx` of 0 identifies the end of the data packet.

The `sidx` -1 mean loss may be present (if non-zero)

sidx	Meaning	Required / optional
-1	numerical integration mean loss	optional

[Return to top](#)

Components

eve

`eve` is an 'event emitter' and its job is to read a list of events from file and send out a subset of events as a binary data stream. It has no standard input and emits a list of `event_ids`, which are 4 byte integers.

`eve` is used to partition lists of events such that a workflow can be distributed across multiple processes.

getmodel

`getmodel` is the component which generates a stream of effective damageability cdfs for a given set of `event_ids` and the impacted exposed items on the basis of their `areaperil_ids` (location) and `vulnerability_ids` (damage function).

gulcalc

`gulcalc` is the component which calculates ground up loss. It takes the `getmodel` output as standard input and based on the sampling parameters specified, performs

Monte Carlo sampling and numerical integration. The output is a stream of ground up loss samples in Oasis kernel format with random samples identified by positive sample indexes (sidx 1 and greater), and special meaning samples assigned to negative sample indexes.

gulcalc also supports the combining and back-allocation of losses arising from multiple subperils impacting the same coverage with some options.

fmcalc

fmcalc is the component which takes the loss stream as standard input and output and applies the policy terms and conditions to produce insured loss samples. fmcalc can be called recursively to perform multiple sequential applications of financial terms (e.g for inuring reinsurance following direct insurance). The output is a table of loss samples in Oasis kernel format, including the (re)insured loss for the numerical integration mean (sidx=-1), and the impacted exposure (sidx=-3).

summarycalc

summarycalc is a component which sums the sampled losses from either gulcalc or fmcalc to the users required level(s) for reporting results. This is a simple sum of the loss value by event_id, sidx and summary_id, where summary_id is a grouping of coverage_id or item_id for gulcalc or output_id for fmcalc defined in the user's input files.

outputcalc

Outputcalc is a general term for an end-of-pipeline component which represents one of a potentially unlimited set of output components. Some examples are provided in the Reference Model. These are;

- eltcalc
- leccalc
- aalc
- pltc
- ordleccalc

The output components generate results such as an event loss table or loss exceedance curve from the sampled output from summarycalc. The output is a results table in csv format or parquet format.

[Return to top](#)

[Go to 4. Reference model](#)

[Back to Contents](#)

4. Reference Model Overview

This section provides an overview of the reference model, which is an implementation of each of the components in the framework.

There are five sub-sections which cover the usage and internal processes of each of the reference components;

- [4.1 Core components](#)
- [4.2 Output components](#)
- [4.3 Data conversion components](#)
- [4.4 Stream conversion components](#)
- [4.5 Validation components](#)

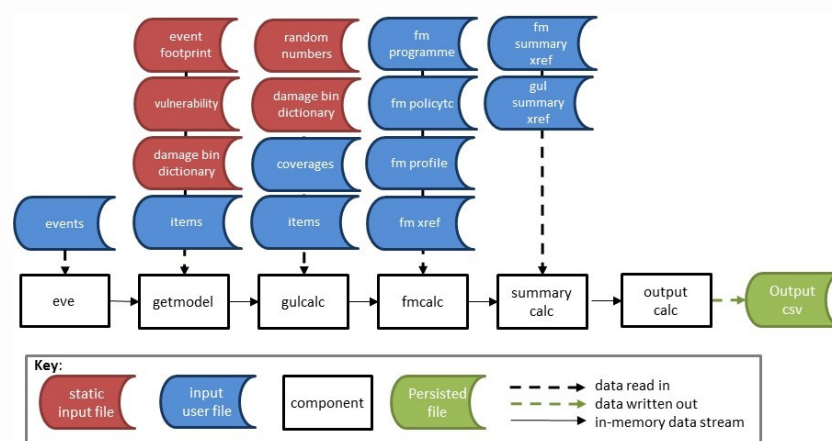
The set of **core components** provided in this release is as follows;

- **eve** is the event distributing utility. Based on the number of events in the input and the number of processes specified as a parameter, eve outputs subsets of the events as a stream. The output streams into getmodel.
- **getmodel** generates a stream of effective damageability cdfs for the input stream of events. It generates cdfs from the model files footprint.bin and vulnerability.bin, and the user's exposures file which is called items.bin. getmodel streams into gulcalc or can be output to a binary file.
- **gulcalc** performs the ground up loss sampling calculations and numerical integration. The output is a stream of sampled ground up losses. This can be output to a binary file or streamed into fmcalt or summarycalc.
- **fmcalt** performs the insurance and reinsurance loss calculations on the ground up loss samples, mean, and total insured value. The output is a stream of loss samples in one of two financial perspectives: net or gross. The result can be output to a binary file or streamed into summarycalc.
- **summarycalc** performs a summing of sampled losses according to the user's reporting requirements. For example this might involve summing coverage losses to regional level, or policy losses to portfolio level. The output is sampled loss by event_id and summary_id, which represents a meaningful group of losses to the user.

The standard input and standard output data streams for the core components are covered in the [Specification](#).

Figure 1 shows the core components workflow and the required data input files.

Figure 1. Core components workflow and required data



The model **static** data for the core workflow, shown as red source files, are the event footprint, vulnerability, damage bin dictionary and random number file. These are stored in the '**static**' sub-directory of the working folder.

The user / analysis **input** data for the core workflow, shown as blue source files, are the events, items, coverages, fm programme, fm policytc, fm profile, fm xref, fm summary xref and gul summary xref files. These are stored in the '**input**' sub-directory of the working folder.

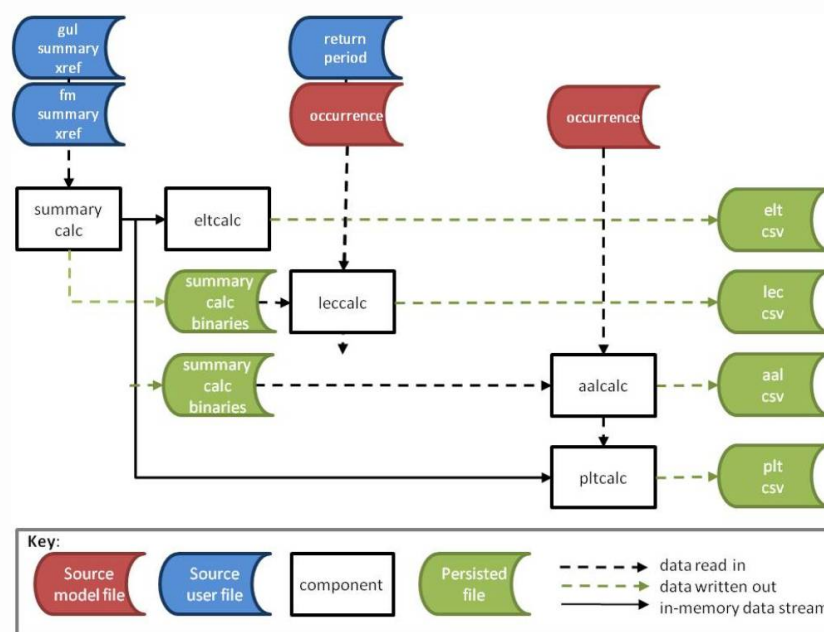
These are all Oasis kernel format data objects with prescribed formats. Note that the events are a user input rather than a static input because the user could choose to run a subset of the full list of events, or even just one event. Usually, though, the whole event set will be run.

The **output components** are various implementations of outputcalc, as described in general terms in the [Specification](#). The results are written directly into csv file as there is no downstream processing.

- **eltcalc** generates an event loss table from the sampled losses from summarycalc. It contains sample mean and standard deviation, and total exposed value for each event at the given summary level.
- **leccalc** generates loss exceedance curve from the sampled losses from summarycalc. There are 8 variants of curves with small differences in the output format but the common output fields are summary_id, return period, and loss exceedance threshold. This output is only available for models which provide an occurrence file.
- **pltcalc** generates a period loss table from the sampled losses from summarycalc. It contains sample mean and standard deviation, and total exposed value for each event and each period (for example a year) at the given summary level. It also contains a date field, corresponding to the date of the event occurrence. This output is only available for models which provide an occurrence file.
- **aalcalc** generates the average annual loss and standard deviation of loss from the sampled losses from summarycalc, for each summary_id. The output also contains total exposed value for each summary level, which is the maximum of the total exposed value across all simulated periods. This output is only available for models which provide an occurrence file.

The files required for the output components are shown in Figure 2.

Figure 2. Output workflows and required data



The **data conversion components** section covers the formats of all of the required data files and explains how to convert data in csv format into binary format, and vice versa.

The **stream conversion components** section explains how to convert the binary data stream output to csv, plus how to convert gulcalc data in csv format into binary format. These components are useful when working with individual components at a more detailed level.

The **validation components** section explains how to use the validation components to check the validity of the static and input files in csv format, before they are converted to binary format. There are both validation checks on individual files and cross checks for consistency across files.

The version of the installed components can be found by using the command line parameter -v. For example;

```
$ gulcalc -v
gulcalc : version: 3.0.7
```

Component usage guidance is available using the parameter -h

```
$ fncalc -h
-a set allocrule (default none)
-M max level (optional)
-p inputpath (relative or full path)
-n feed net value (used for reinsurance)
-O Alloc rule2 optimization off
-d debug
-v version
-h help
```

The components have additional command line parameters depending on their particular function. These are described in detail in the following pages.

[Return to top](#)

[Go to 4.1 Core Components section](#)

[Back to Contents](#)

4.1 Core Components

eve

eve takes a list of event ids in binary format as input and generates a partition of event ids as a binary data stream, according to the parameters supplied. Events are "shuffled" by being assigned to processes one by one cyclically, rather than being distributed to processes in blocks, in the order they are input. This helps to even the workload by process, in case all the "big" events are together in the same range of the event list.

Output stream

The output stream is a simple list of event_ids (4 byte integers).

Parameters

Required parameters are;

- process number - specifies the process number to receive a partition of events
- number of processes - determines the total number of partitions of events to distribute to processes

Optional parameters are;

- -n - No shuffled events. Events are split up and distributed in blocks in respect of the order they are input.
- -r - Randomise events. Events are split up and distributed in blocks randomly using the Fisher-Yates shuffle. No shuffled events takes priority in cases where both optional parameters are supplied.

Usage

```
$ eve [parameters] > [output].bin
$ eve [parameters] | getmodel | gulcalc [parameters] > [stdout].bin
```

Example

```
$ eve 1 2 > events1_2_shuffled.bin
$ eve -n 1 2 > events1_2_unshuffled.bin
$ eve -r 1 2 > events1_2_random.bin
$ eve 1 2 | getmodel | gulcalc -r -S100 -i - > gulcalc1_2.bin
```

In this example, the events from the file events.bin will be read into memory and the first half (partition 1 of 2) would be streamed out to binary file, or downstream to a single process calculation workflow.

Internal data

The program requires an event binary. The file is picked up from the input sub-directory relative to where the program is invoked and has the following filename;

- input/events.bin

The data structure of events.bin is a simple list of event ids (4 byte integers).

[Return to top](#)

getmodel

getmodel generates a stream of effective damageability distributions (cdfs) from an input list of events. Specifically, it combines the probability distributions from the model files, footprint.bin and vulnerability.bin, to generate effective damageability cdfs for the subset of exposures contained in the items.bin file and converts them into a binary stream.

This is reference example of the class of programs which generates the damage distributions for an event set and streams them into memory. It is envisaged that model developers who wish to use the toolkit as a back-end calculator of their

existing platforms can write their own version of getmodel, reading in their own source data and converting it into the standard output stream. As long as the standard input and output structures are adhered to, each program can be written in any language and read any input data.

Output stream

Byte 1	Bytes 2-4	Description
0	1	cdf stream

Parameters

None

Usage

```
$ [stdin component] | getmodel | [stdout component]
$ [stdin component] | getmodel > [stdout].bin
$ getmodel < [stdin].bin > [stdout].bin
```

Example

```
$ eve 1 1 | getmodel | gulcalc -r -S100 -i gulcalci.bin
$ eve 1 1 | getmodel > getmodel.bin
$ getmodel < events.bin > getmodel.bin
```

Internal data

The program requires the footprint binary and index file for the model, the vulnerability binary model file, and the items file representing the user's exposures. The files are picked up from sub-directories relative to where the program is invoked, as follows;

- static/footprint.bin
- static/footprint.idx
- static/vulnerability.bin
- static/damage_bin_dict.bin
- input/items.bin

The getmodel output stream is ordered by event and streamed out in blocks for each event.

Calculation

The program filters the footprint binary file for all areaperil_id's which appear in the items file. This selects the event footprints that impact the exposures on the basis on their location. Similarly the program filters the vulnerability file for vulnerability_id's that appear in the items file. This selects conditional damage distributions which are relevant for the exposures.

The intensity distributions from the footprint file and conditional damage distributions from the vulnerability file are convolved for every combination of areaperil_id and vulnerability_id in the items file. The effective damage probabilities are calculated, for each damage bin, by summing the product of conditional damage probabilities with intensity probabilities for each event, areaperil, vulnerability combination across the intensity bins.

The resulting discrete probability distributions are converted into discrete cumulative distribution functions 'cdfs'. Finally, the damage bin mid-point from the damage bin dictionary ('interpolation' field) is read in as a new field in the cdf stream as 'bin_mean'. This field is the conditional mean damage for the bin and it is used to choose the interpolation method for random sampling and numerical integration calculations in the gulcalc component.

[Return to top](#)

gulcalc

The gulcalc program performs Monte Carlo sampling of ground up loss by randomly sampling the cumulative probability of damage from the uniform distribution and generating damage factors by interpolation of the random numbers against the effective damage cdf. Other loss metrics are computed and assigned to special meaning sample index values as descibed below.

The sampling methodologies are linear interpolation, quadratic interpolation and point value sampling depending on the damage bin definitions in the input data.

Gulcalc also performs back-allocation of total coverage losses to the contributing subperil item losses (for multi-subperil models). This occurs when there are two or more items representing losses from different subperils to the same coverage, such as wind loss and storm surge loss, for example. In these cases, because the subperil losses are generated independently from each other it is possible to result in a total damage ratio greater than 1 for the coverage, or a total loss greater than the Total Insured Value "TIV". Back-allocation ensures that the total loss for a coverage cannot exceed the input TIV.

Stream output

Byte 1	Bytes 2-4	Description
2	1	loss stream

Parameters

Required parameters are;

- -S{number}. Number of samples
- -a{number} -i{destination}

The destination is either a filename or named pipe, or use - for standard output.

Optional parameters are;

- -L{number} loss threshold (optional) excludes losses below the threshold from the output stream
- -d debug mode - output random numbers rather than losses (optional)
- -A automatically hashed seed driven random number generation (default)
- -R{number} Number of random numbers to generate dynamically
- -s{number} Manual seed for random numbers (to make them repeatable)
- -r Read random numbers from random.bin file

Usage

```
$ [stdin component] | gulcalc [parameters] | [stdout component]
$ [stdin component] | gulcalc [parameters]
$ gulcalc [parameters] < [stdin].bin
```

Example

```
$ eve 1 1 | getmodel | gulcalc -R1000000 -S100 -a1 -i - | fmcalt > fmcalt.bin
$ eve 1 1 | getmodel | gulcalc -R1000000 -S100 -a1 -i - | summarycalc -i -1 summarycalc1.bin
$ eve 1 1 | getmodel | gulcalc -r -S100 -a1 -i gulcalci.bin
$ gulcalc -r -S100 -i -a1 gulcalci.bin < getmodel.bin
```

Internal data

The program requires the damage bin dictionary binary for the static folder and the item and coverage binaries from the input folder. The files are found in the following locations relative to the working directory with the filenames;

- static/damage_bin_dictionary.bin
- input/items.bin
- input/coverages.bin

If the user specifies -r as a parameter, then the program also picks up a random number file from the static directory. The filename is;

- static/random.bin

Calculation

The stdin stream is a block of cdfs which are ordered by event_id, areaperil_id, vulnerability_id and bin_index ascending, from getmodel. The gulcalc program constructs a cdf for each item, based on matching the areaperil_id and vulnerability_id from the stdin and the item file.

Random sampling

Random samples are indexed using positive integers starting from 1, called the 'sidx', or sample index.

For each item cdf and for the number of samples specified, the program draws a uniformly distributed random number and uses it to sample ground up loss from the cdf using one of three methods, as follows;

For a given damage interval corresponding to a cumulative probability interval that each random number falls within;

- If the conditional mean damage (of the cdf) is the mid-point of the damage bin interval (of the damage bin dictionary) then the gulcalc program performs linear interpolation.
- If the conditional mean damage is equal to the lower and upper damage threshold of the damage bin interval (i.e the bin represents a point value, not a range) then that point value is sampled.
- Else, the gulcalc program performs quadrature interpolation using the bin_mean to calculate the quadratic equation in the damage interval. The bin mean must lie within the middle third of the damage interval to generate a valid quadrature cdf. This requirement comes from the necessary condition that the probability density function must be ≥ 0 across the entire damage interval.

An example of the three cases and methods is given below;

bin_from	bin_to	bin_mean	Method used
0.1	0.2	0.15	Linear interpolation
0.1	0.1	0.1	Sample bin value
0.1	0.2	0.14	Quadratic interpolation

If the -R parameter is used along with a specified number of random numbers then random numbers used for sampling are generated on the fly for each event and group of items which have a common group_id using the Mersenne twister pseudo random number generator (the default RNG of the C++ v11 compiler). These random numbers are not repeatable, unless a seed is also specified (-s{number}).

If the -r parameter is used, gulcalc reads a random number from the provided random number file, which produces repeatable results.

The default random number behaviour (no additional parameters) is to generate random numbers from a seed determined by a combination of the event_id and group_id, which produces repeatable results. See [Random Numbers](#) for more details.

Each sampled damage is multiplied by the item TIV, looked up from the coverage file.

Special samples

Samples with negative indexes have special meanings as follows;

sidx	description
-1	Numerical integration mean
-2	Numerical integration standard deviation
-3	Impacted exposure
-4	Chance of loss
-5	Maximum loss

- The numerical integration mean loss, `sidx=-1`, is computed by multiplying the item TIV looked up from the coverage file with the mean damage. The mean damage is computed by a sum across all damage bins of the product of the damage bin mean and the probability density (the difference between consecutive damage bin `prob_to` values).
- The numerical integration standard deviation of loss, `sidx=-2`, is computed by multiplying the item TIV looked up from the coverage file with the standard deviation of damage. The standard deviation of damage is the sqrt of the sum across all damage bins of the product of the squared errors between the numerical integration mean damage and each damage bin mean with the probability density (the difference between consecutive damage bin `prob_to` values).
- The impacted exposure, `sidx=-3` represents the 100% damage scenario to all items impacted by (within the footprint of) an event. It is 1 multiplied by the item TIV looked up from the coverage file.
- The chance of loss, `sidx -4`, is the probability that, conditional on the event occurring, the damage/loss is greater than zero. This value is computed directly from the damage distribution. Its value is 1 if the upper threshold of the first damage bin is non-zero (meaning no chance of zero damage), else it is 1 - `prob_to` of the first damage bin.
- The maximum loss, `sidx -5`, represents the maximum possible loss computed from the damage distribution. This is the upper damage threshold of the first damage bin which has `prob_to = 1`, multiplied by the item TIV from the coverage file.

Allocation method

The allocation method determines how item losses are adjusted when a coverage is subject to losses from multiple perils, because the total loss to a coverage from multiple perils cannot exceed the input TIV. This situation is identified when multiple item_ids in the item file share the same coverage_id. The TIV is held in the coverages file against the coverage_id and the item_id TIV is looked up from its relationship to coverage_id in the item file.

The allocation methods are as follows;

a	description
0	Pass losses through unadjusted (used for single peril models)
1	Sum the losses and cap them to the TIV. Back-allocate TIV to the contributing items in proportion to the unadjusted losses
2	Keep the maximum subperil loss and set the others to zero. Back-allocate equally when there are equal maximum losses

The mean, impacted exposure and maximum loss special samples are also subject to these allocation rules. The impacted exposure value, sidx -3, is always back-allocated equally to the items, for allocation rules 1 and 2, since by definition it is the same value for all items related to the same coverage.

[Return to top](#)

fmcalc

fmcalc is the reference implementation of the Oasis Financial Module. It applies policy terms and conditions to the ground up losses and produces loss sample output. It reads in the loss stream from either gulcalc or from another fmcalc and can be called recursively and apply several consecutive sets of policy terms and conditions.

Stream output

Byte 1	Bytes 2-4	Description
2	1	loss stream

Parameters

Optional parameters are;

- p {string} The location of the input files. The default location is the 'input' directory in the present working directory
- a{integer} The back allocation rule to apply. The options are 0 (no allocation), 1 (ground up loss basis) or 2 (prior level loss basis). The default is 0.
- n Output net losses. Net losses are the difference between the input loss and the calculated loss. The default is to output the calculated loss.
- S Use fm_profile_step input file (default is to use fm_profile)

Usage

```
$ [stdin component] | fmcalc [parameters] | [stdout component]
$ [stdin component] | fmcalc [parameters] > [stdout].bin
$ fmcalc [parameters] < [stdin].bin > [stdout].bin
```

Example

```
$ eve 1 1 | getmodel | gulcalc -r -S100 -a1 -i - | fmcalc -p direct -a2 | summarycalc -f -2 - | eltc
> elt.csv
$ eve 1 1 | getmodel | gulcalc -r -S100 -a1 -i - | fmcalc -p direct -a1 > fmcalc.bin
$ fmcalc -p ri1 -a2 -S -n < gulcalci.bin > fmcalc.bin
$ fmcalc -p direct | fmcalc -p ri1 -n | fmcalc -p ri2 -n < gulcalci.bin > fm_ri2_net.bin
```

Internal data

For the gulcalc item stream input, the program requires the item, coverage and fm input data files, which are Oasis abstract data objects that describe an insurance or reinsurance programme. This data is picked up from the following files relative to the working directory by default;

- input/items.bin
- input/coverages.bin
- input/fm_programme.bin
- input/fm_policytc.bin
- input/fm_profile.bin or fm_profile_step.bin
- input/fm_xref.bin

For loss stream input from either gulcalc or fmcalc, the program requires only the four fm input data files,

- input/fm_programme.bin
- input/fm_policytc.bin
- input/fm_profile.bin or fm_profile_step.bin
- input/fm_xref.bin

The location of the files can be changed by using the -p parameter followed by the path location relative to the present working directory. eg -p ri1

Calculation

fmcalc passes the loss samples, including the numerical integration mean, sidx -1, and impacted exposure, sidx -3, through a set of financial calculations which are defined by the input files. The special samples -2, -4 and -5 are ignored and dropped in the output. For more information about the calculation see [Financial Module](#)

[Return to top](#)

summarycalc

The purpose of summarycalc is firstly to aggregate the samples of loss to a level of interest for reporting, thereby reducing the volume of data in the stream. This is a generic first step which precedes all of the downstream output calculations. Secondly, it unifies the formats of the gulcalc and fmcalc streams, so that they are transformed into an identical stream type for downstream outputs. Finally, it can generate up to 10 summary level outputs in one go, creating multiple output streams or files.

The output is similar to the gulcalc or fmcalc input which are losses are by sample index and by event, but the ground up or (re)insurance loss input losses are grouped to an abstract level represented by a summary_id. The relationship between the input identifier and the summary_id are defined in cross reference files called **gulsummaryxref** and **fmsummaryxref**.

Stream output

Byte 1	Bytes 2-4	Description
3	1	summary stream

Parameters

The input stream should be identified explicitly as -i input from gulcalc or -f input from fmcalc.

summarycalc supports up to 10 concurrent outputs. This is achieved by explicitly directing each output to a named pipe, file, or to standard output.

For each output stream, the following tuple of parameters must be specified for at least one summary set;

- summaryset_id number. valid values are -0 to -9
- destination pipe or file. Use either - for standard output, or {name} for a named pipe or file.

For example the following parameter choices are valid;

```
$ summarycalc -i -1 -
'outputs results for summaryset 1 to standard output
$ summarycalc -i -1 summarycalc1.bin
'outputs results for summaryset 1 to a file (or named pipe)
$ summarycalc -i -1 summarycalc1.bin -2 summarycalc2.bin
'outputs results for summaryset 1 and 2 to a file (or named pipe)
```

Note that the summaryset_id relates to a summaryset_id in the required input data file **gulsummaryxref.bin** or **fmsummaryxref.bin** for a gulcalc input stream or a fmcalc input stream, respectively, and represents a user specified summary reporting level. For example summaryset_id = 1 represents portfolio level, summaryset_id = 2 represents zipcode level and summaryset_id 3 represents site level.

Usage

```
$ [stdin component] | summarycalc [parameters] | [stdout component]
$ [stdin component] | summarycalc [parameters]
$ summarycalc [parameters] < [stdin].bin
```

Example

```
$ eve 1 1 | getmodel | gulcalc -r -S100 -a1 -i - | summarycalc -i -1 - | eltcals > eltcals.csv
$ eve 1 1 | getmodel | gulcalc -r -S100 -a1 -i - | summarycalc -i -1 gulsummarycalc.bin
$ eve 1 1 | getmodel | gulcalc -r -S100 -a1 -i - | fmcals | summarycalc -f -1 fmsummarycalc.bin
$ summarycalc -f -1 fmsummarycalc.bin < fmcals.bin
```

Internal data

The program requires the gulsummaryxref file for gulcalc input (-i option), or the fmsummaryxref file for fmcals input (-f option). This data is picked up from the following files relative to the working directory;

- input/gulsummaryxref.bin
- input/fmsummaryxref.bin

Calculation

summarycalc takes either ground up loss from gulcalc or financial loss samples from fmcals as input and aggregates them to a user-defined summary reporting level. The output is similar to the input, individual losses by sample index and by event, but the ground up or financial losses are summed to an abstract level represented by a summary_id. The relationship between the input identifier, item_id for gulcalc or output_id for fmcals, and the summary_id are defined in the input files.

The special samples are computed as follows;

- The numerical integration mean, sidx -1, impacted exposure, sidx -3, maximum loss value, sidx -5, are treated as normal samples and summed to each summary_id.
- The numerical integration standard deviation, sidx -2, is dropped.
- From gulcalc input only, the chance of loss, sidx -4, at a given summary level is aggregated using the law of total probability, evaluated as $1 - \text{probability all items under a summary_id have zero loss}$. $1 - (1-C1)(1-C2)...(1-CN)$ where C1, C2, ..CN represents the chance of loss for item 1,2,..N under each summary_id, which are present in the gulcalc stream.

[Return to top](#)

[Go to 4.2 Output Components section](#)

[Back to Contents](#)

4.2 Output Components

eltcalc

The program calculates mean and standard deviation of loss by summary_id and by event_id.

Parameters

None

Usage

```
$ [stdin component] | eltcalc > elt.csv  
$ eltcalc < [stdin].bin > elt.csv
```

Example

```
$ eve 1 1 | getmodel | gulcalc -r -S100 -c - | summarycalc -g -1 - | eltcalc > elt.csv  
$ eltcalc < summarycalc.bin > elt.csv
```

Internal data

No additional data is required, all the information is contained within the input stream.

Calculation

For each summary_id and event_id, the sample mean and standard deviation is calculated from the sampled losses in the summarycalc stream and output to file. The analytical mean is also output as a separate record, differentiated by a 'type' field. The exposure_value, which is carried in the event_id, summary_id header of the stream is also output.

Output

csv file with the following fields;

Name	Type	Bytes	Description	Example
summary_id	int	4	summary_id representing a grouping of losses	10
type	int	4	1 for analytical mean, 2 for sample mean	2
event_id	int	4	Oasis event_id	45567
mean	float	4	mean	1345.678
standard_deviation	float	4	sample standard deviation	945.89
exposure_value	float	4	exposure value for summary_id affected by the event	70000

[Return to top](#)

leccalc

Loss exceedance curves, also known as exceedance probability curves, are computed by a rank ordering a set of losses by period and computing the probability of exceedance for each level of loss based on relative frequency. Losses are first assigned to periods of time (typically years) by reference to the **occurrence** file which contains the event occurrences in each period over a timeline of, say, 10,000 periods. Event losses are summed within each period for an aggregate loss exceedance curve, or the maximum of the event losses in each period is taken for an occurrence loss exceedance curve. From this point, there are a few variants available as follows;

- Wheatsheaf/multiple EP - losses by period are rank ordered for each sample, which produces many loss exceedance curves - one for each sample across the same timeline. The wheatsheaf shows the variation in return period loss due to sampled damage uncertainty, for a given timeline of occurrences.

- Full uncertainty/single EP - all sampled losses by period are rank ordered to produce a single loss exceedance curve. This treats each sample as if it were another period of losses in an extrapolated timeline. Stacking the curves end-to-end rather than viewing side-by-side as in the wheatsheaf is a form of averaging with respect to a particular return period loss and provides stability in the point estimate, for a given timeline of occurrences.
- Sample mean - the losses by period are first averaged across the samples, and then a single loss exceedance curve is created from the period sample mean losses.
- Wheatsheaf mean - the loss exceedance curves from the Wheatsheaf are averaged across each return period, which produces a single loss exceedance curve.

The ranked losses represent the first, second, third, etc.. largest loss periods within the total number of periods of say 10,000 years. The relative frequency of these periods of loss is interpreted as the probability of loss exceedance, that is to say that the top ranked loss has an exceedance probability of 1 in 10000, or 0.01%, the second largest loss has an exceedance probability of 0.02%, and so on. In the output file, the exceedance probability is expressed as a return period, which is the reciprocal of the exceedance probability multiplied by the total number of periods. Only non-zero loss periods are returned.

Parameters

- -K{sub-directory}. The subdirectory of /work containing the input summarycalc binary files. Then the following tuple of parameters must be specified for at least one analysis type;
- Analysis type. Use -F for Full Uncertainty Aggregate, -f for Full Uncertainty Occurrence, -W for Wheatsheaf Aggregate, -w for Wheatsheaf Occurrence, -S for Sample Mean Aggregate, -s for Sample Mean Occurrence, -M for Mean of Wheatsheaf Aggregate, -m for Mean of Wheatsheaf Occurrence
- Output filename

An optional parameter is;

- -r. Use return period file - use this parameter if you are providing a file with a specific list of return periods. If this file is not present then all calculated return periods will be returned, for losses greater than zero.

Usage

```
$ leccalc [parameters] > lec.csv
```

Examples

```
'First generate summarycalc binaries by running the core workflow, for the required summary set
$ eve 1 2 | getmodel | gulcalc -r -S100 -c - | summarycalc -g -1 - >
work/summary1/summarycalc1.bin
$ eve 2 2 | getmodel | gulcalc -r -S100 -c - | summarycalc -g -1 - >
work/summary1/summarycalc2.bin
'Then run leccalc, pointing to the specified sub-directory of work containing summarycalc
binaries.
$ leccalc -Ksummary1 -F lec_full_uncertainty_agg.csv -f lec_full_uncertainty_occ.csv
'With return period file
$ leccalc -r -Ksummary1 -F lec_full_uncertainty_agg.csv -f lec_full_uncertainty_occ.csv
```

Internal data

leccalc requires the occurrence.bin file

- input/occurrence.bin

and will optionally use the following additional files if present

- input/returnperiods.bin
- input/periods.bin

leccalc does not have a standard input that can be streamed in. Instead, it reads in summarycalc binary data from a file in a fixed location. The format of the binaries must match summarycalc standard output. The location is in the 'work' subdirectory of the present working directory. For example;

- work/summarycalc1.bin
- work/summarycalc2.bin
- work/summarycalc3.bin

The user must ensure the work subdirectory exists. The user may also specify a subdirectory of /work to store these files. e.g.

- work/summaryset1/summarycalc1.bin
- work/summaryset1/summarycalc2.bin
- work/summaryset1/summarycalc3.bin

The reason for leccalc not having an input stream is that the calculation is not valid on a subset of events, i.e. within a single process when the calculation has been distributed across multiple processes. It must bring together all event losses before assigning event losses to periods and ranking losses by period. The summarycalc losses for all events (all processes) must be written to the /work folder before running leccalc.

Calculation

All files with extension .bin from the specified subdirectory are read into memory, as well as the occurrence.bin. The summarycalc losses are grouped together and sampled losses are assigned to period according to which period the events occur in.

If multiple events occur within a period;

- For **aggregate** loss exceedance curves, the sum of losses is calculated.
- For **occurrence** loss exceedance curves, the maximum loss is calculated.

Then the calculation differs by lec type, as follows;

- Full uncertainty - all losses by period are rank ordered to produce a single loss exceedance curve.
- Wheatsheaf - losses by period are rank ordered for each sample, which produces many loss exceedance curves - one for each sample across the same timeline.
- Sample mean - the losses by period are first averaged across the samples, and then a single loss exceedance curve is created from the period sample mean losses.
- Wheatsheaf mean - the return period losses from the Wheatsheaf are averaged, which produces a single loss exceedance curve.

For all curves, the analytical mean loss (sidx = -1) is output as a separate exceedance probability curve. If the calculation is run with 0 samples, then leccalc will still return the analytical mean loss exceedance curve. The 'type' field in the output identifies the type of loss exceedance curve, which is 1 for analytical mean, and 2 for curves calculated from the samples.

Output

csv file with the following fields;

Full uncertainty, Sample mean and Wheatsheaf mean loss exceedance curve

Name	Type	Bytes	Description	Example
summary_id	int	4	summary_id representing a grouping of losses	10
type	int	4	1 for analytical mean, 2 for sample mean	2
return_period	float	4	return period interval	250
loss	float	4	loss exceedance threshold for return period	546577.8

Wheatsheaf loss exceedance curve

Name	Type	Bytes	Description	Example
summary_id	int	4	summary_id representing a grouping of losses	10
sidx	int	4	Oasis sample index	50
return_period	float	4	return period interval	250
loss	float	4	loss exceedance threshold for return period	546577.8

Period weightings

An additional feature of leccalc is available to vary the relative importance of the period losses by providing a period weightings file to the calculation. In this file, a weight can be assigned to each period make it more or less important than neutral weighting (1 divided by the total number of periods). For example, if the neutral weight for period 1 is 1 in 10000 years, or 0.0001, then doubling the weighting to 0.0002 will mean that period's loss reoccurrence rate would double. Assuming no other period losses, the return period of the loss of period 1 in this example would be halved.

All period_nos must appear in the file from 1 to P (no gaps). There is no constraint on the sum of weights. Periods with zero weight will not contribute any losses to the loss exceedance curve.

This feature will be invoked automatically if the periods.bin file is present in the input directory.

[Return to top](#)

pltcalc

The program outputs sample mean and standard deviation by summary_id, event_id and period_no. The analytical mean is also output as a separate record, differentiated by a 'type' field. It also outputs an event occurrence date.

Parameters

None

Usage

```
$ [stdin component] | pltcalc > plt.csv  
$ pltcalc < [stdin].bin > plt.csv
```

Example

```
$ eve 1 1 | getmodel | gulcalc -r -S100 -C1 | summarycalc -1 - | pltcalc > plt.csv  
$ pltcalc < summarycalc.bin > plt.csv
```

Internal data

pltcalc requires the occurrence.bin file

- input/occurrence.bin

Calculation

The occurrence.bin file is read into memory. For each summary_id, event_id and period_no, the sample mean and standard deviation is calculated from the sampled losses in the summarycalc stream and output to file. The exposure_value, which is carried in the event_id, summary_id header of the stream is also output, as well as the date field(s) from the occurrence file.

Output

There are two output formats, depending on whether an event occurrence date is an integer offset to some base date that most external programs can interpret as a real date, or a calendar day in a numbered scenario year. The output format will depend on the format of the date fields in the occurrence.bin file.

In the former case, the output format is;

Name	Type	Bytes	Description	Example
type	int	4	1 for analytical mean, 2 for sample mean	1
summary_id	int	4	summary_id representing a grouping of losses	10
event_id	int	4	Oasis event_id	45567
period_no	int	4	identifying an abstract period of time, such as a year	56876
mean	float	4	mean	1345.678
standard_deviation	float	4	sample standard deviation	945.89
exposure_value	float	4	exposure value for summary_id affected by the event	70000
date_id	int	4	the date_id of the event occurrence	28616

Using a base date of 1/1/1900 the integer 28616 is interpreted as 16/5/1978.

In the latter case, the output format is;

Name	Type	Bytes	Description	Example
type	int	4	1 for analytical mean, 2 for sample mean	1
summary_id	int	4	summary_id representing a grouping of losses	10
event_id	int	4	Oasis event_id	45567
period_no	int	4	identifying an abstract period of time, such as a year	56876
mean	float	4	mean	1345.678
standard_deviation	float	4	sample standard deviation	945.89
exposure_value	float	4	exposure value for summary_id affected by the event	70000
occ_year	int	4	the year number of the event occurrence	56876
occ_month	int	4	the month of the event occurrence	5
occ_year	int	4	the day of the event occurrence	16

[Return to top](#)

aalcalc

aalcalc computes the overall average annual loss and standard deviation of annual loss.

Two types of aal and standard deviation of loss are calculated; analytical (type 1) and sample (type 2). If the analysis is run with zero samples, then only type 1 statistics are returned by aalcalc.

The Average Loss Convergence Table 'ALCT' is a second optional output which can be generated from aalcalc. This provides extra statistical output which can be used to estimate the amount of simulation error in the average annual loss estimate from samples (type 2).

Internal data

aalcalc requires the occurrence.bin file

- input/occurrence.bin

aalcalc does not have a standard input that can be streamed in. Instead, it reads in summarycalc binary data from a file in a fixed location. The format of the binaries must match summarycalc standard output. The location is in the 'work' subdirectory of the present working directory. For example;

- work/summarycalc1.bin
- work/summarycalc2.bin
- work/summarycalc3.bin

The user must ensure the work subdirectory exists. The user may also specify a subdirectory of /work to store these files. e.g.

- work/summaryset1/summarycalc1.bin
- work/summaryset1/summarycalc2.bin
- work/summaryset1/summarycalc3.bin

The reason for aalcalc not having an input stream is that the calculation is not valid on a subset of events, i.e. within a single process when the calculation has been distributed across multiple processes. It must bring together all event losses before assigning event losses to periods and finally computing the final statistics.

Parameters

- -K{sub-directory}. The sub-directory of /work containing the input aalcalc binary files.
- -o -l {confidence_level} -c {file path} - Generates the alct report in addition to the aalcalc report (optional)

Usage

```
$ aalcalc [parameters] > aal.csv
```

Examples


```

First generate summarycalc binaries by running the core workflow, for the required summary set
$ eve 1 2 | getmodel | gulcalc -r -S100 -c - | summarycalc -g -1 - >
work/summary1/summarycalc1.bin
$ eve 2 2 | getmodel | gulcalc -r -S100 -c - | summarycalc -g -1 - >
work/summary1/summarycalc2.bin
Then run aalcalc, pointing to the specified sub-directory of work containing summarycalc
binaries.
$ aalcalc -Ksummary1 > aal.csv
Add alct output at 95% confidence level
$ aalcalc -Ksummary1 -o -l 0.95 -c alct.csv > aal.csv

```

Output

AAL:

csv file containing the following fields;

Name	Type	Bytes	Description	Example
summary_id	int	4	summary_id representing a grouping of losses	1
type	int	4	1 for analytical statistics, 2 for sample statistics	2
mean	float	8	average annual loss	1014.23
standard_deviation	float	8	standard deviation of annual loss	11039.78

ALCT:

csv file containing the following fields;

Name	Type	Bytes	Description	Example
SummaryId	int	4	summary_id representing a grouping of losses	1
MeanLoss	float	4	the average annual loss estimate from samples	1014.23
SDLoss	float	8	the standard deviation of annual loss from samples	11039.78
SampleSize	int	8	the number of samples used to produce the statistics	100
LowerCI	float	8	the lower threshold of the confidence interval for the mean estimate	1004.52
UpperCI	float	8	the upper threshold of the confidence interval for the mean estimate	1023.94
StandardError	float	8	the total standard error of the mean estimate	5.90
RelativeError	float	8	the StandardError divided by the mean estimate	0.005
VarElementHaz	float	8	the contribution to variance of the estimate from the hazard	8707.40
StandardErrorHaz	float	8	the square root of VarElementHaz	93.31
RelativeErrorHaz	float	8	the StandardErrorHaz divided by the mean estimate	0.092
VarElementVuln	float	8	the contribution to variance of the estimate from the vulnerability	34.81
StandardErrorVuln	float	8	the square root of VarElementVuln	5.90
RelativeErrorVuln	float	8	the StandardErrorVuln divided by the mean estimate	0.005

Calculation

The occurrence file and summarycalc files from the specified subdirectory are read into memory. Event losses are assigned to periods based on when the events occur and summed by period and by sample. These are referred to as 'annual loss samples'.

AAL calculation:

For type 1, calculations are performed on the type 1 (numerically integrated) mean annual losses by period. The AAL is the mean annual losses summed across the periods and divided by the number of periods. The standard deviation is the square root of the sum of squared errors between each annual mean loss and the AAL mean divided by the degrees of freedom (periods - 1).

For type 2 the mean and standard deviation of the annual loss samples are calculated across all samples and periods. The mean estimates the average annual loss, calculated as the sum of all annual loss samples divided by the total number of periods times the number of samples. The standard deviation is the square root of the sum of squared errors between each annual loss sample and the type 2 mean, divided by the degrees of freedom (periods × samples - 1).

ALCT calculation:

In ALCT, MeanLoss and SDLoss are the same as the type 2 mean and standard deviation from the AAL report. StandardError indicates how much the average annual loss estimate might vary if the simulation were rerun with different random numbers, reflecting the simulation error in the estimate. RelativeError, the StandardError as a percentage of the mean, is convenient for assessing simulation error and acceptable levels are typically expressed in percentage terms. StandardError is derived from the ANOVA metrics described below.

LowerCI and UpperCI represent the absolute lower and upper thresholds for the confidence interval for the AAL estimate, indicating the range of losses within a specified confidence level. A higher confidence level results in a wider confidence interval.

Variance components:

VarElementHaz and VarElementVuln arise from attributing variance in the annual loss to hazard effects (variation due to event intensity across years) and vulnerability effects (variation due to sampling from exposure's damage uncertainty distributions). This is done using a one-factor effects model and standard analysis of variance 'ANOVA' on the annual loss samples.

In the one-factor model, annual loss in year i and sample m , denoted $L(i,m)$, is expressed as:

$$L(i,m) = AAL + h(i) + v(i,m)$$

where;

- $i = 1, 2, \dots$ to I years (periods)
- $m = 1, 2, \dots$ to M samples
- $h(i)$ = mean loss for year i - AAL
- $v(i,m) = L(i,m)$ - mean loss for year i
- I is the total number of years (periods) in the model
- M is the number of samples specified at runtime by the user

Total variance in annual loss is partitioned into independent hazard and vulnerability effects:

$$\text{Var}(L) = \text{Var}(h) + \text{Var}(v)$$

ANOVA is used to estimate the variance components $\text{Var}(h)$ and $\text{Var}(v)$. For standard Oasis models, since the events are fixed across years, the simulation error in the AAL estimate arises only from the vulnerability component.

The StandardError of the AAL estimate in ALCT follows from the calculation of the Variance of the AAL estimate as follows;

$$\text{Var}(\text{AAL estimate}) = \text{VarElementVuln} = \text{Var}(v) / (I * M)$$

$$\text{StandardErrorVuln} = \sqrt{\text{VarElementVuln}}$$

$$\text{StandardError} = \text{StandardErrorVuln}$$

Finally, ALCT provides statistics for multiple increasing sample subsets, showing the convergence of the AAL estimate with increasing sample sizes. These subsets are non-overlapping and fixed, starting with SampleSize=1 ($m=1$), SampleSize=2 ($m=2$ to 3), SampleSize=4 ($m=4$ to 7), up to the maximum subset size. The final row gives statistics for the total samples M , using all available samples.

Period weightings

An additional feature of aalcalc is available to vary the relative importance of the period losses by providing a period weightings file to the calculation. In this file, a weight can be assigned to each period make it more or less important than neutral weighting (1 divided by the total number of periods). For example, if the neutral weight for period 1 is 1 in 10000 years, or 0.0001, then doubling the weighting to 0.0002 will mean that period's loss reoccurrence rate would double and the loss contribution to the average annual loss would double.

All period_nos must appear in the file from 1 to P (no gaps). There is no constraint on the sum of weights. Periods with zero weight will not contribute any losses to the AAL.

This feature will be invoked automatically if the periods.bin file is present in the input

directory.

[Return to top](#)

Parameters

Optional parameter for aalcalc;

- -c {file path} - Generate the alct report

kat

In cases where events have been distributed to multiple processes, the output files can be concatenated to standard output.

Parameters

Optional parameters are:

- -d {file path} - The directory containing output files to be concatenated.
- -s - Sort by event ID (currently only supported for eltcalf output).

The sort by event ID option assumes that events have not been distributed to processes randomly and the list of event IDs in events.bin is sequential and contiguous. Should either of these conditions be false, the output will still contain all events but sorting cannot be guaranteed.

Usage

```
$ kat [parameters] [file]... > [stdout component]
```

Examples

```
$ kat -d pltcalc_output/ > pltcalc.csv
$ kat eltcalf_P1 eltcalf_P2 eltcalf_P3 > eltcalf.csv
$ kat -s eltcalf_P1 eltcalf_P2 eltcalf_P3 > eltcalf.csv
$ kat -s -d eltcalf_output/ > eltcalf.csv
```

Files are concatenated in the order in which they are presented on the command line. Should a file path be specified, files are concatenated in alphabetical order. When asked to sort by event ID, the order of input files is irrelevant.

[Return to top](#)

katparquet

The output parquet files from multiple processes can be concatenated to a single parquet file. The results are automatically sorted by event ID. Unlike [kat](#), the [ORD table name](#) for the input files must be specified on the command line.

Parameters

- -d {file path} - The directory containing output files to be concatenated.
- -M - Concatenate MPLT files
- -Q - Concatenate QPLT files
- -S - Concatenate SPLT files
- -m - Concatenate MELT files
- -q - Concatenate QELT files
- -s - Concatenate SELT files
- -o {filename} - Output concatenated file

Usage

```
$ katparquet [parameters] -o [filename.parquet] [file]...
```

Examples

```
$ katparquet -d mplt_files/ -M -o MPLT.parquet
$ katparquet -q -o QPLT.parquet qplt_P1.parquet qplt_P2.parquet qplt_P3.parquet
```

[Return to top](#)

[Go to 4.3 Data conversion components section](#)

[Back to Contents](#)

4.4 Data conversion components

The following components convert input data in csv format to the binary format required by the calculation components in the reference model;

Static data

- **aggregatevulnerabilitytobin** converts the aggregate vulnerability data.
- **damagebintobin** converts the damage bin dictionary.
- **footprinttobin** converts the event footprint.
- **lossfactorstobin** converts the lossfactors data.
- **randtobin** converts a list of random numbers.
- **vulnerabilitytobin** converts the vulnerability data.
- **weightstobin** converts the weights data.

A reference intensity bin dictionary csv should also exist, although there is no conversion component for this file because it is not needed for calculation purposes.

Input data

- **amplificationtobin** converts the amplifications data.
- **coveragetobin** converts the coverages data.
- **ensembletobin** converts the ensemble data.
- **evetobin** converts a list of event_ids.
- **itemtobin** converts the items data.
- **gulsummaryxreftobin** converts the gul summary xref data.
- **fmpolicytctobin** converts the fm policytc data.
- **fmprogrammetobin** converts the fm programme data.
- **fmprofiletobin** converts the fm profile data.
- **fmsummaryxreftobin** converts the fm summary xref data.
- **fmxreftobin** converts the fm xref data.
- **occurrencetobin** converts the event occurrence data.
- **returnperiodtobin** converts a list of return periods.
- **periodstobin** converts a list of weighted periods (optional).
- **quantiletobin** converts a list of quantiles (optional).

These components are intended to allow users to generate the required input binaries from csv independently of the original data store and technical environment. All that needs to be done is first generate the csv files from the data store (SQL Server database, etc).

The following components convert the binary input data required by the calculation components in the reference model into csv format;

Static data

- **aggregatevulnerabilitytocsv** converts the aggregate vulnerability data.
- **damagebintocsv** converts the damage bin dictionary.
- **footprinttocsv** converts the event footprint.
- **lossfactorstocsv** converts the lossfactors data.
- **randtocsv** converts a list of random numbers.
- **vulnerabilitytocsv** converts the vulnerability data.
- **weightstocsv** converts the weights data.

Input data

- **amplificationtocsv** converts the amplifications data.
- **coveragetocsv** converts the coverages data.
- **ensembletocsv** converts the ensemble data.
- **evetocsv** converts a list of event_ids.
- **itemtocsv** converts the items data.
- **gulsummaryxreftocsv** converts the gul summary xref data.
- **fmpolicytctocsv** converts the fm policytc data.
- **fmprogrammetocsv** converts the fm programme data.
- **fmprofiletocsv** converts the fm profile data.
- **fmsummaryxreftocsv** converts the fm summary xref data.
- **fmxreftocsv** converts the fm xref data.
- **occurrencetocsv** converts the event occurrence data.
- **returnperiodtocsv** converts a list of return periods.
- **periodstocsv** converts a list of weighted periods (optional).
- **quantiletocsv** converts a list of quantiles (optional).

These components are provided for the convenience of viewing the data and debugging.

Static data

aggregate vulnerability

The aggregate vulnerability file is required for the gulmc component. It contains the conditional distributions of damage for each intensity bin and for each vulnerability_id. This file must have the following location and filename;

- static/aggregate_vulnerability.bin

File format

The csv file should contain the following fields and include a header row.

Name	Type	Bytes	Description	Example
aggregate_vulnerability_id	int	4	Oasis vulnerability_id	45
vulnerability_id	int	4	Oasis vulnerability_id	45

If this file is present, the weights.bin or weights.csv file must also be present. The data should not contain nulls.

aggregatevulnerabilitytobin

```
$ aggregatevulnerabilitytobin < aggregate_vulnerability.csv > aggregate_vulnerability.bin
```

aggregatevulnerabilitytocsv

```
$ aggregatevulnerabilitytocsv < aggregate_vulnerability.bin > aggregate_vulnerability.csv
```

[Return to top](#)

damage bin dictionary

The damage bin dictionary is a reference table in Oasis which defines how the effective damageability cdfs are discretized on a relative damage scale (normally between 0 and 1). It is required by getmodel and gulcalc and must have the following location and filename;

- static/damage_bin_dict.bin

File format

The csv file should contain the following fields and include a header row.

Name	Type	Bytes	Description	Example
bin_index	int	4	Identifier of the damage bin	1
bin_from	float	4	Lower damage threshold for the bin	0.01
bin_to	float	4	Upper damage threshold for the bin	0.02
interpolation	float	4	Interpolation damage value for the bin (usually the mid-point)	0.015

The interval_type field has been deprecated and will be filled with zeros in the binary file. It does not need to be included as the final column in the csv file:

Name	Type	Bytes	Description	Example
interval_type	int	4	Identifier of the interval type, e.g. closed, open (deprecated)	0

The data should be ordered by bin_index ascending and not contain nulls. The bin_index should be a contiguous sequence of integers starting from 1.

damagebintobin

```
$ damagebintobin < damage_bin_dict.csv > damage_bin_dict.bin
```

Validation checks on the damage bin dictionary csv file are conducted by default during conversion to binary format. These can be suppressed with the -N argument:

```
$ damagebintobin -N < damage_bin_dict.csv > damage_bin_dict.bin
```

damagebintocsv

```
$ damagebintocsv < damage_bin_dict.bin > damage_bin_dict.csv
```

The deprecated `interval_type` field can be sent to the output using the `-i` argument:

```
$ damagebintocsv -i < damage_bin_dict.bin > damage_bin_dict.csv
```

[Return to top](#)

intensity bin dictionary

The intensity bin dictionary defines the meaning of the bins of the hazard intensity measure. The hazard intensity measure could be flood depth, windspeed, peak ground acceleration etc, depending on the type of peril. The range of hazard intensity values in the model is discretized into bins, each with a unique and contiguous `bin_index` listed in the intensity bin dictionary. The `bin_index` is used as a reference in the footprint file (field `intensity_bin_index`) to specify the hazard intensity for each event and areaperil.

This file is for reference only as it is not used in the calculation so there is no component to convert it to binary format.

The csv file should contain the following fields and include a header row.

Name	Type	Bytes	Description	Example
bin_index	int	4	Identifier of the intensity bin	1
bin_from	float	4	Lower intensity threshold for the bin	56
bin_to	float	4	Upper intensity threshold for the bin	57
interpolation	float	4	Mid-point intensity value for the bin	0.015
interval_type	int	4	Identifier of the interval type, e.g. closed, open	1

The data should be ordered by `bin_index` ascending and not contain nulls. The `bin_index` should be a contiguous sequence of integers starting from 1.

[Return to top](#)

footprint

The event footprint is required for the `getmodel` component, as well as an index file containing the starting positions of each event block. These must have the following location and filenames;

- `static/footprint.bin`
- `static/footprint.idx`

File format

The csv file should contain the following fields and include a header row.

Name	Type	Bytes	Description	Example
event_id	int	4	Oasis event_id	1
areaperil_id	int	4	Oasis areaperil_id	4545
intensity_bin_index	int	4	Identifier of the intensity bin	10
prob	float	4	The probability mass for the intensity bin between 0 and 1	0.765

The data should be ordered by `event_id`, `areaperil_id` and not contain nulls.

footprinttobin

```
$ footprinttobin -i {number of intensity bins} < footprint.csv
```

This command will create a binary file `footprint.bin` and an index file `footprint.idx` in the working directory. The number of intensity bins is the maximum value of `intensity_bin_index`.

Validation checks on the footprint csv file are conducted by default during conversion to binary format. These can be suppressed with the `-N` argument:

```
$ footprinttobin -i {number of intensity bins} -N < footprint.csv > footprint.bin
```

There is an additional parameter `-n`, which should be used when there is only one record per `event_id` and `areaperil_id`, with a single `intensity_bin_index` value and `prob = 1`. This is the special case 'no hazard intensity uncertainty'. In this case, the usage is as follows.

```
$ footprinttobin -i {number of intensity bins} -n < footprint.csv
```

Both parameters -i and -n are held in the header of the footprint.bin and used in getmodel.

The output binary and index file names can be explicitly set using the -b and -x flags respectively:

```
$ footprinttobin -i {number of intensity bins} -b {output footprint binary file name} -x {output footprint index file name} < footprint.csv
```

Both output binary and index file names must be given to use this option.

In the case of very large footprint files, it may be preferable to compress the data as it is written to the binary file. Compression is performed using `zlib` by issuing the -z flag. If the -u flag is used in addition, the index file will include the uncompressed data size. It is recommended to use the -u flag to prevent any memory issues during decompression with getmodel or footprinttocsv.

```
$ footprinttobin -i {number of intensity bins} -z < footprint.csv
$ footprinttobin -i {number of intensity bins} -z -u < footprint.csv
```

The value of the -u parameter is held in the same location as -n in the header of the footprint.bin file, left-shifted by 1.

footprinttocsv

```
$ footprinttocsv > footprint.csv
```

footprinttocsv requires a binary file footprint.bin and an index file footprint.idx to be present in the working directory.

Input binary and index file names can be explicitly set using the -b and -x flags respectively:

```
$ footprinttocsv -b {input footprint binary file name} -x {input footprint index file name} > footprint.csv
```

Both input binary and index file name must be given to use this option.

Footprint binary files that contain compressed data require the -z argument to be issued:

```
$ footprinttocsv -z > footprint.csv
```

[Return to top](#)

Loss Factors

The lossfactors binary maps the event_id/amplification_id pairs with post loss amplification factors, and is supplied by the model providers. The first 4 bytes are preserved for future use and the data format is as follows. It is required by Post Loss Amplification (PLA) workflow must have the following location and filename;

- static/lossfactors.bin

File format

The csv file should contain the following fields and include a header row.

Name	Type	Bytes	Description	Example
event_id	int	4	Event ID	1
count	int	4	Number of amplification IDs associated with the event ID	1
amplification_id	int	4	Amplification ID	1
factor	float	4	The uplift factor	1.01

All fields must not have null values. The csv file will not contain the count, and the conversion tools will add/remove this count.

lossfactorstobin

```
$ lossfactorstobin < lossfactors.csv > lossfactors.bin
```

lossfactorstocsv

```
$ lossfactorstocsv < lossfactors.bin > lossfactors.csv
```

[Return to top](#)

Random numbers

A random number file may be provided for the gulcalc component as an option (using gulcalc -r parameter) The random number binary contains a list of random numbers used for ground up loss sampling in the kernel calculation. It must have the following location and filename;

- static/random.bin

If the gulcalc -r parameter is not used, the random number binary is not required and random numbers are instead generated dynamically during the calculation, using the -R parameter to specify how many should be generated.

The random numbers can be imported from a csv file using the component randtobin.

File format

The csv file should contain a simple list of random numbers and include a header row.

Name	Type	Bytes	Description	Example
rand	float	4	Number between 0 and 1	0.75875

randtobin

```
$ randtobin < random.csv > random.bin
```

randtocsv

There are a few parameters available which allow the generation of a random number csv file as follows;

- -r convert binary float input to csv
- -g generate random numbers {number of random numbers}
- -S seed value {seed value}

```
$ randtocsv -r < random.bin > random.csv
$ randtocsv -g 1000000 > random.csv
$ randtocsv -g 1000000 -S 1234 > random.csv
```

The -S {seed value} option produces repeatable random numbers, whereas usage of -g alone will generate a different set every time.

[Return to top](#)

vulnerability

The vulnerability file is required for the getmodel component. It contains the conditional distributions of damage for each intensity bin and for each vulnerability_id. This file must have the following location and filename;

- static/vulnerability.bin

File format

The csv file should contain the following fields and include a header row.

Name	Type	Bytes	Description	Example
vulnerability_id	int	4	Oasis vulnerability_id	45
intensity_bin_index	int	4	Identifier of the hazard intensity bin	10
damage_bin_index	int	4	Identifier of the damage bin	20
prob	float	4	The probability mass for the damage bin	0.186

The data should be ordered by vulnerability_id, intensity_bin_index and not contain nulls.

vulnerabilitytobin

```
$ vulnerabilitytobin -d {number of damage bins} < vulnerability.csv > vulnerability.bin
```

The parameter -d number of damage bins is the maximum value of

damage_bin_index. This is held in the header of vulnerability.bin and used by getmodel.

Validation checks on the vulnerability csv file are conducted by default during conversion to binary format. These can be suppressed with the -N argument:

```
$ vulnerabilitytobin -d {number of damage bins} -N < vulnerability.csv > vulnerability.bin
```

In the case of very large vulnerability files, it may be preferable to create an index file to improve performance. Issuing the -i flag creates vulnerability.bin and vulnerability.idx in the current working directory:

```
$ vulnerabilitytobin -d {number of damage bins} -i < vulnerability.csv
```

Additionally, the data can be compressed as it is written to the binary file. Compression is performed with [zlib](#) by issuing the -z flag. This creates vulnerability.bin.z and vulnerability.idx.z in the current working directory:

```
$ vulnerabilitytobin -d {number of damage bins} -i < vulnerability.csv
```

The getmodel component will look for the presence of index files in the following order to determine which algorithm to use to extract data from vulnerability.bin:

- 1. static/vulnerability.idx.z
- 2. static/vulnerability.idx

vulnerabilitytocsv

```
$ vulnerabilitytocsv < vulnerability.bin > vulnerability.csv
$ vulnerabilitytocsv -i > vulnerability.csv
$ vulnerabilitytocsv -z > vulnerability.csv
```

[Return to top](#)

Weights

The vulnerability weights binary contains the the weighting of each vulnerability function in all areaperil IDs. The data format is as follows. It is required by gulmc with the aggregate_vulnerability file and must have the following location and filename;

- static/weights.bin

File format

The csv file should contain the following fields and include a header row.

Name	Type	Bytes	Description	Example
areaperil_id	int	4	Areaperil ID	1
vulnerability_id	int	4	Vulnerability ID	1
weight	float	4	The weighting factor	1.0

All fields must not have null values.

weightstobin

```
$ weightstobin < weights.csv > weights.bin
```

weightstocsv

```
$ weightstocsv < weights.bin > weights.csv
```

[Return to top](#)

Input data

Amplifications

The amplifications binary contains the list of item IDs mapped to amplification IDs. The data format is as follows. It is required by Post Loss Amplification (PLA) workflow must have the following location and filename;

- input/amplifications.bin

File format

The csv file should contain the following fields and include a header row.

Name	Type	Bytes	Description	Example
item_id	int	4	Item ID	1
amplification_id	int	4	Amplification ID	1

The item_id must start from 1 and must be contiguous and not have null values. The binary file only contains the amplification IDs and assumes the item_ids would start from 1 and are contiguous.

amplificationtobin

```
$ amplificationtobin < amplifications.csv > amplifications.bin
```

amplificationtocsv

```
$ amplificationtocsv < amplifications.bin > amplifications.csv
```

[Return to top](#)

Coverages

The coverages binary contains the list of coverages and the coverage TIVs. The data format is as follows. It is required by gulcalc and fmcalc and must have the following location and filename;

- input/coverages.bin

File format

The csv file should contain the following fields and include a header row.

Name	Type	Bytes	Description	Example
coverage_id	int	4	Identifier of the coverage	1
tiv	float	4	The total insured value of the coverage	200000

Coverage_id must be an ordered contiguous sequence of numbers starting at 1.

coveragetobin

```
$ coveragetobin < coverages.csv > coverages.bin
```

coveragetocsv

```
$ coveragetocsv < coverages.bin > coverages.csv
```

[Return to top](#)

ensemble

The ensemble file is used for ensemble modelling (multiple views) which maps sample IDs to particular ensemble ID groups. It is an optional file for use with AAL and LEC. It must have the following location and filename;

- input/ensemble.bin

File format

The csv file should contain a list of event_ids (integers) and include a header.

Name	Type	Bytes	Description	Example
sidx	int	4	Sample ID	1
ensemble_id	int	4	Ensemble ID	1

ensembletobin

```
$ ensembletobin < ensemble.csv > ensemble.bin
```

ensembletocsv

```
$ ensembletocsv < ensemble.bin > ensemble.csv
```

[Return to top](#)

events

One or more event binaries are required by eve. It must have the following location and filename;

- input/events.bin

File format

The csv file should contain a list of event_ids (integers) and include a header.

Name	Type	Bytes	Description	Example
event_id	int	4	Oasis event_id	4545

evetobin

```
$ evetobin < events.csv > events.bin
```

evetocsv

```
$ evetocsv < events.bin > events.csv
```

[Return to top](#)

items

The items binary contains the list of exposure items for which ground up loss will be sampled in the kernel calculations. The data format is as follows. It is required by gulcalc and outputcalc and must have the following location and filename;

- input/items.bin

File format

The csv file should contain the following fields and include a header row.

Name	Type	Bytes	Description	Example
item_id	int	4	Identifier of the exposure item	1
coverage_id	int	4	Identifier of the coverage	3
areaperil_id	int	4	Identifier of the locator and peril	4545
vulnerability_id	int	4	Identifier of the vulnerability distribution	645
group_id	int	4	Identifier of the correlaton group	3

The data should be ordered by areaperil_id, vulnerability_id ascending and not contain nulls. item_id must be a contiguous sequence of numbers starting from 1.

itemtobin

```
$ itemtobin < items.csv > items.bin
```

itemtocsv

```
$ itemtocsv < items.bin > items.csv
```

[Return to top](#)

gul summary xref

The gulsummaryxref binary is a cross reference file which determines how item or coverage losses from gulcalc output are summed together into at various summary levels in summarycalc. It is required by summarycalc and must have the following location and filename;

- input/gulsummaryxref.bin

File format

The csv file should contain the following fields and include a header row.

Name	Type	Bytes	Description	Example
item_id / coverage_id	int	4	Identifier of the item or coverage	3
summary_id	int	4	Identifier of the summary level grouping	3
summaryset_id	int	4	Identifier of the summary set	1

- The first field must be consistent with the corresponding index in the ground up loss stream. If it is loss stream (stream_id 2/1) then item_id must be used. If it is gulcalc coverage stream (stream_id 1/2) then coverage_id must be used.
- The data should not contain nulls and there should be at least one summary set in the file.
- Valid entries for summaryset_id is all integers between 0 and 9 inclusive.
- Within each summary set, all coverages_id's from the coverages file must be present.

One summary set consists of a common summaryset_id and each item_id being assigned a summary_id. An example is as follows.

item_id	summary_id	summaryset_id
1	1	1
2	1	1
3	1	1
4	2	1
5	2	1
6	2	1

This shows, for summaryset_id=1, items 1-3 being grouped into summary_id = 1 and items 4-6 being grouped into summary_id = 2. This could be an example of a 'site' level grouping, for example. The summary_ids should be held in a dictionary which contains the description of the ids to make meaning of the output results. For instance;

summary_id	summaryset_id	summary_desc
1	1	site_435
2	1	site_958

This cross reference information is not required in ktools.

Up to 10 summary sets may be provided in gulsummaryxref, depending on the required summary reporting levels for the analysis. Here is an example of the 'site' summary level with summaryset_id=1, plus an 'account' summary level with summaryset_id = 2. In summary set 2, the account summary level includes both sites because all items are assigned a summary_id of 1.

item_id	summary_id	summaryset_id
1	1	1
2	1	1
3	1	1
4	2	1
5	2	1
6	2	1
1	1	2
2	1	2
3	1	2
4	1	2
5	1	2
6	1	2

gulsummaryxreftobin

```
$ gulsummaryxreftobin < gulsummaryxref.csv > gulsummaryxref.bin
```

gulsummaryxreftocsv

```
$ gulsummaryxreftocsv < gulsummaryxref.bin > gulsummaryxref.csv
```

[Return to top](#)

The fm programme binary file contains the level heirarchy and defines aggregations of losses required to perform a loss calculation, and is required for fmcalf only.

This must have the following location and filename;

- input/fm_programme.bin

File format

The csv file should contain the following fields and include a header row.

Name	Type	Bytes	Description	Example
from_agg_id	int	4	Oasis Financial Module from_agg_id	1
level_id	int	4	Oasis Financial Module level_id	1
to_agg_id	int	4	Oasis Financial Module to_agg_id	1

- All fields must have integer values and no nulls
- Must have at least one level, starting from level_id = 1, 2 3 ...
- For level_id = 1, the set of values in from_agg_id must be equal to the set of item_ids in the input ground up loss stream (which has fields event_id, item_id, idx, gul). Therefore level 1 always defines a group of items.
- For subsequent levels, the from_agg_id must be the distinct values from the previous level to_agg_id field.
- The from_agg_id and to_agg_id values, for each level, should be a contiguous block of integers (a sequence with no gaps). This is not a strict rule in this version and it will work with non-contiguous integers, but it is recommended as best practice.

fmprogrammetobin

```
$ fmprogrammetobin < fm_programme.csv > fm_programme.bin
```

fmprogrammetocsv

```
$ fmprogrammetocsv < fm_programme.bin > fm_programme.csv
```

[Return to top](#)

fm profile

The fmprofile binary file contains the list of calculation rules with profile values (policytc_ids) that appear in the policytc file. This is required for fmcalf only.

There are two versions of this file and either one or the other can be used at a time.

- fm_profile - original set of fields
- fm_profile_step - extended set of fields to include step policy fields

They must be in the following location with filename formats;

- input/fm_profile.bin
- input/fm_profile_step.bin

File format

The csv file should contain the following fields and include a header row.

fm_profile

Name	Type	Bytes	Description	Example
policytc_id	int	4	Primary key	34
calcrule_id	int	4	The calculation rule that applies to the terms	12
deductible_1	int	4	First deductible	0.03
deductible_2	float	4	Second deductible	50000
deductible_3	float	4	Third deductible	100000
attachment_1	float	4	Attachment point, or excess	1000000
limit_1	float	4	Limit	5000000
share_1	float	4	First proportional share	0.8
share_2	float	4	Second proportional share	0.25
share_3	float	4	Third proportional share	1

fm_profile_step

Name	Type	Bytes	Description	Example
policytc_id	int	4	Primary key	34
calcrule_id	int	4	The calculation rule that applies to the terms	12
deductible_1	int	4	First deductible	0.03
deductible_2	float	4	Second deductible	50000
deductible_3	float	4	Third deductible	100000
attachment_1	float	4	Attachment point, or excess	1000000
limit_1	float	4	First limit	5000000
share_1	float	4	First proportional share	0.8
share_2	float	4	Second proportional share	0.25
share_3	float	4	Third proportional share	1
step_id	int	4	Step number	1
trigger_start	float	4	Start trigger for payout	0.05
trigger_end	float	4	End trigger for payout	0.15
payout_start	float	4	Start payout	100
payout_end	float	4	End payout	200
limit_2	float	4	Second limit	3000000
scale_1	float	4	Scaling (inflation) factor 1	0.03
scale_2	float	4	Scaling (inflation) factor 2	0.2

- A reference table listing the valid values for calcrule_id and which of the fields are required is available in Appendix B FM Profiles.
- All distinct policytc_id values that appear in the policytc table must appear once in the policytc_id field of the profile table. We suggest that policytc_id=1 is included by default using calcrule_id = 100 and all fields = 0 as a default 'null' calculation rule whenever no terms and conditions apply to a particular level_id / agg_id in the policytc table.
- Any fields that are not required for the profile should be set to zero.

fmprofiletobin

```
$ fmprofiletobin < fm_profile.csv > fm_profile.bin  
$ fmprofiletobin -S < fm_profile_step.csv > fm_profile_step.bin
```

fmprofiletocsv

```
$ fmprofiletocsv < fm_profile.bin > fm_profile.csv  
$ fmprofiletocsv -S < fm_profile_step.bin > fm_profile_step.csv
```

[Return to top](#)

fm_policytc

The fm_policytc binary file contains the cross reference between the aggregations of losses defined in the fm programme file at a particular level and the calculation rule that should be applied as defined in the fm profile file. This file is required for fmcalf only.

This must have the following location and filename;

- input/fm_policytc.bin

File format

The csv file should contain the following fields and include a header row.

Name	Type	Bytes	Description	Example
layer_id	int	4	Oasis Financial Module layer_id	1
level_id	int	4	Oasis Financial Module level_id	1
agg_id	int	4	Oasis Financial Module agg_id	1
policytc_id	int	4	Oasis Financial Module policytc_id	1

- All fields must have integer values and no nulls
- Must contain the same levels as the fm programme where level_id = 1, 2, 3 ...
- For every distinct combination of to_agg_id and level_id in the programme table, there must be a corresponding record matching level_id and agg_id values in the policytc table with a valid value in the policytc_id field.
- layer_id = 1 at all levels except the last where there may be multiple layers, with

layer_id = 1, 2, 3 ... This allows for the specification of several policy contracts applied to the same aggregation of losses defined in the programme table.

***fm*policytctobin**

```
$ fmpolicytctobin < fm_policytc.csv > fm_policytc.bin
```

***fm*policytctocsv**

```
$ fmpolicytctocsv < fm_policytc.bin > fm_policytc.csv
```

[Return to top](#)

fm summary xref

The fm summary xref binary is a cross reference file which determines how losses from fmcalt output are summed together at various summary levels by summarycalc. It is required by summarycalc and must have the following location and filename;

- input/fmsummaryxref.bin

File format

The csv file should contain the following fields and include a header row.

Name	Type	Bytes	Description	Example
output_id	int	4	Identifier of the coverage	3
summary_id	int	4	Identifier of the summary level group for one or more output losses	1
summaryset_id	int	4	Identifier of the summary set (0 to 9 inclusive)	1

- The data should not contain nulls and there should be at least one summary set in the file.
- Valid entries for summaryset_id is all integers between 0 and 9 inclusive.
- Within each summary set, all output_id's from the fm xref file must be present.

One summary set consists of a common summaryset_id and each output_id being assigned a summary_id. An example is as follows.

output_id	summary_id	summaryset_id
1	1	1
2	2	1

This shows, for summaryset_id=1, output_id=1 being assigned summary_id = 1 and output_id=2 being assigned summary_id = 2.

If the output_id represents a policy level loss output from fmcalt (the meaning of output_id is defined in the fm xref file) then no further grouping is performed by summarycalc and this is an example of a 'policy' summary level grouping.

Up to 10 summary sets may be provided in this file, depending on the required summary reporting levels for the analysis. Here is an example of the 'policy' summary level with summaryset_id=1, plus an 'account' summary level with summaryset_id = 2. In summary set 2, the 'account' summary level includes both policy's because both output_id's are assigned a summary_id of 1.

output_id	summary_id	summaryset_id
1	1	1
2	2	1
1	1	2
2	1	2

If a more detailed summary level than policy is required for insured losses, then the user should specify in the fm profile file to back-allocate fmcalt losses to items. Then the output_id represents back-allocated policy losses to item, and in the fmsummaryxref file these can be grouped into any summary level, such as site, zipcode, line of business or region, for example. The user needs to define output_id in the fm xref file, and group them together into meaningful summary levels in the fm summary xref file, hence these two files must be consistent with respect to the meaning of output_id.

fmsummaryxreftobin

```
$ fmsummaryxreftobin < fmsummaryxref.csv > fmsummaryxref.bin
```

fmsummaryxreftocsv

```
$ fmsummaryxreftocsv < fmsummaryxref.bin > fmsummaryxref.csv
```

[Return to top](#)

fm xref

The fmxref binary file contains cross reference data specifying the output_id in the fmcalf as a combination of agg_id and layer_id, and is required by fmcalf.

This must be in the following location with filename format;

- input/fm_xref.bin

File format

The csv file should contain the following fields and include a header row.

Name	Type	Bytes	Description	Example
output_id	int	4	Identifier of the output group of losses	1
agg_id	int	4	Identifier of the agg_id to output	1
layer_id	int	4	Identifier of the layer_id to output	1

The data should not contain any nulls.

The output_id represents the summary level at which losses are output from fmcalf, as specified by the user.

There are two cases;

- losses are output at the final level of aggregation (represented by the final level to_agg_id's from the fm programme file) for each contract or layer (represented by the final level layer_id's in the fm policytc file)
- losses are back-allocated to the item level and output by item (represented by the from_agg_id of level 1 from the fm programme file) for each policy contract / layer (represented by the final level layer_id's in the fm policytc file)

For example, say there are two policy layers (with layer_ids=1 and 2) which applies to the sum of losses from 4 items (the summary level represented by agg_id=1). Without back-allocation, the policy summary level of losses can be represented as two output_id's as follows;

output_id	agg_id	layer_id
1	1	1
2	1	2

If the user wants to back-allocate policy losses to the items and output the losses by item and policy, then the item-policy summary level of losses would be represented by 8 output_id's, as follows;

output_id	agg_id	layer_id
1	1	1
2	2	1
3	3	1
4	4	1
5	1	2
6	2	2
7	3	2
8	4	2

The fm summary xref file must be consistent with respect to the meaning of output_id in the fmxref file.

fmxreftobin

```
$ fmxreftobin < fm_xref.csv > fm_xref.bin
```

fmxreftocsv


```
$ fmxreftocsv < fm_xref.bin > fm_xref.csv
```

[Return to top](#)

occurrence

The occurrence file is required for certain output components which, in the reference model, are leccalc, pltcac and aalcalc. In general, some form of event occurrence file is required for any output which involves the calculation of loss metrics over a period of time. The occurrence file assigns occurrences of the event_ids to numbered periods. A period can represent any length of time, such as a year, or 2 years for instance. The output metrics such as mean, standard deviation or loss exceedance probabilities are with respect to the chosen period length. Most commonly in catastrophe modelling, the period of interest is a year.

The occurrence file also includes date fields.

- occ_year, occ_month, occ_day. These are all integers representing occurrence year, month and day.
- occ_hour, occ_monute. These are optional and are all integers representing occurrence hour and minute.

File format

The csv file should contain the following fields and include a header row.

Name	Type	Bytes	Description	Example
event_id	int	4	The occurrence event_id	45567
period_no	int	4	A numbered period in which the event occurs	56876
occ_year	int	4	the year number of the event occurrence	56876
occ_month	int	4	the month of the event occurrence	5
occ_day	int	4	the day of the event occurrence	16

The occurrence year in this example is a scenario numbered year, which cannot be expressed as a real date in a standard calendar.

In addition, the following fields are optional and should comprise the sixth and seventh column respectively:

Name	Type	Bytes	Description	Example
occ_hour	int	4	The hour of the event occurrence	13
occ_minute	int	4	The minute of the event occurrence	52

The date fields are converted to a single number through an algorithm for efficient storage in the binary file. The data type for this field is either an integer when the optional date fields are not included or a long long integer when the these date fields are included. This should not be confused with the deprecated occ_date_id field.

occurrenceetobin

A required parameter is -P, the total number of periods of event occurrences. The total number of periods is held in the header of the binary file and used in output calculations.

```
$ occurrenceetobin -P10000 < occurrence.csv > occurrence.bin
```

If it is desirable to include the occ_hour and occ_minute fields in the binary file, the -H argument should be given. A flag to signify the presence of these fields is set in the header of the binary file, which is read by other kiools components. If these fields do not exist in the csv file, both are assigned the value of 0 when written to the binary file.

```
$ occurrenceetobin -P10000 -H < occurrence.csv > occurrence.bin
```

occurrenceetocsv

```
$ occurrenceetocsv < occurrence.bin > occurrence.csv
```

[Return to top](#)

return period

The returnperiods binary file is a list of return periods that the user requires to be

included in loss exceedance curve (leccalc) results.

This must be in the following location with filename format;

- input/returnperiods.bin

File format

The csv file should contain the following field and include a header.

Name	Type	Bytes	Description	Example
return_period	int	4	Return period	250

returnperiodtobin

```
$ returnperiodtobin < returnperiods.csv > returnperiods.bin
```

returnperiodtocsv

```
$ returnperiodtocsv < returnperiods.bin > returnperiods.csv
```

[Return to top](#)

periods

The periods binary file is a list of all the periods that are in the model and is optional for weighting the periods in the calculation. The file is used in the calculation of the loss exceedance curve (leccalc) and aalcalc results.

This must be in the following location with filename format;

- input/periods.bin

File format

The csv file should contain the following field and include a header.

Name	Type	Bytes	Description	Example
period_no	int	4	A numbered period in which the event occurs	4545
weight	int	4	relative weight to P, the maximum period_no	0.0003

All periods must be present in this file (no gaps in period_no from 1 to P).

periodstobin

```
$ periodstobin < periods.csv > periods.bin
```

periodstocsv

```
$ periodstocsv < periods.bin > periods.csv
```

[Return to top](#)

Quantile

The quantile binary file contains a list of user specified quantile floats. The data format is as follows. It is optionally used by the Quantile Event/Period Loss tables and must have the following location and filename;

- input/quantile.bin

File format

The csv file should contain the following fields and include a header row.

Name	Type	Bytes	Description	Example
quantile	float	4	Quantile float	0.1

All fields must not have null values.

quantileto bin

```
$ quantileto bin < quantile.csv > quantile.bin
```

quantileto csv

```
$ quantileto csv < quantile.bin > quantile.csv
```

[Return to top](#)

[Go to 4.5 Stream conversion components section](#)

[Back to Contents](#)

4.5 Stream conversion components

The following components convert the binary output of each calculation component to csv format;

- **cdftocsv** is a utility to convert binary format CDFs to a csv. getmodel standard output can be streamed directly into cdftocsv, or a binary file of the same format can be piped in.
- **gultocsv** is a utility to convert binary format GULs to a csv. gulcalc standard output can be streamed directly into gultocsv, or a binary file of the same format can be piped in.
- **fmtocsv** is a utility to convert binary format losses to a csv. fmcalc standard output can be streamed directly into fmtocsv, or a binary file of the same format can be piped in.
- **summarycalctocsv** is a utility to convert binary format summarycalc losses to a csv. summarycalc standard output can be streamed directly into summarycalctocsv, or a binary file of the same format can be piped in.

Additionally, the following component is provided to convert csv data into binary format;

- **gultobin** is a utility to convert gulcalc data in csv format into binary format such that it can be piped into fmcalc.

Figure 1 shows the workflows for the binary stream to csv conversions.

Figure 1. Stream conversion workflows

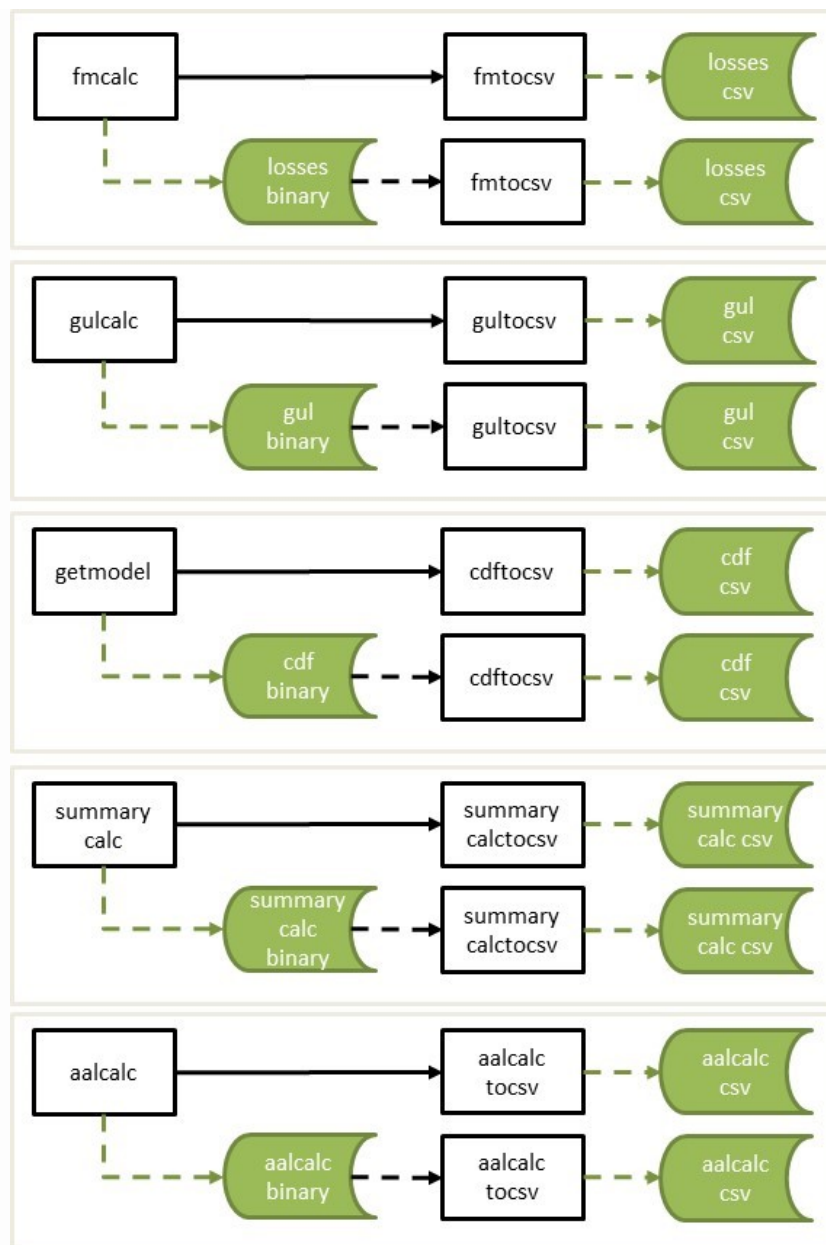
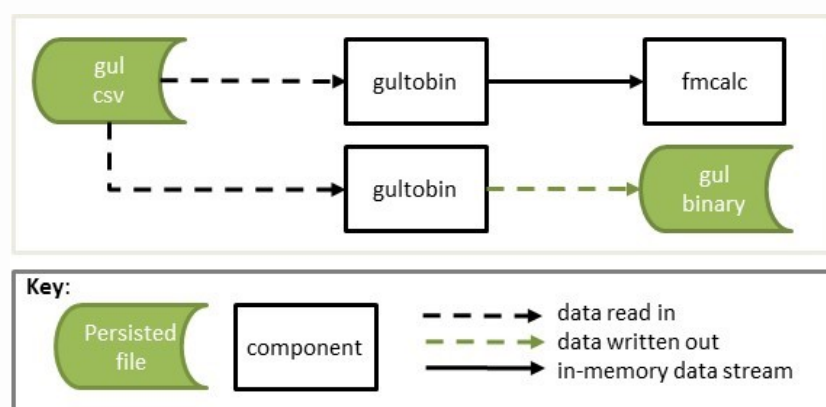


Figure 2 shows the workflows for the gultobin component.

Figure 2. gultobin workflows



cdftocsv

A component which converts the `getmodel` output stream, or binary file with the same structure, to a csv file.

Standard input stream

Byte 1	Bytes 2-4	Description
0	1	cdf

A binary file of the same format can be piped into cdfotcsv.

Usage

```
$ [stdin component] | cdfotcsv > [output].csv
$ cdfotcsv < [stdin].bin > [output].csv
```

Example

```
$ eve 1 1 | getmodel | cdfotcsv > cdf.csv
$ cdfotcsv < getmodel.bin > cdf.csv
```

Output

Csv file with the following fields;

Name	Type	Bytes	Description	Example
event_id	int	4	Oasis event_id	4545
areaperil_id	int	4	Oasis areaperil_id	345456
vulnerability_id	int	4	Oasis vulnerability_id	345
bin_index	int	4	Damage bin index	20
prob_to	float	4	The cumulative probability at the upper damage bin threshold	0.765
bin_mean	float	4	The conditional mean of the damage bin	0.45

[Return to top](#)

gultotcsv

A component which converts the gulcalc item or coverage stream, or binary file with the same structure, to a csv file.

Standard input stream

Byte 1	Bytes 2-4	Description
1	1	gulcalc item
1	2	gulcalc coverage

A binary file of the same format can be piped into gultotcsv.

Usage

```
$ [stdin component] | gultotcsv > [output].csv
$ gultotcsv < [stdin].bin > [output].csv
```

Example

```
$ eve 1 1 | getmodel | gulcalc -r -S100 -c - | gultotcsv > gulcalcc.csv
$ gultotcsv < gulcalci.bin > gulcalci.csv
```

Output

Csv file with the following fields;

gulcalc item stream 1/1

Name	Type	Bytes	Description	Example
event_id	int	4	Oasis event_id	4545
item_id	int	4	Oasis item_id	300
sidx	int	4	Sample index	10
loss	float	4	The ground up loss value	5675.675

gulcalc coverage stream 1/2

Name	Type	Bytes	Description	Example
event_id	int	4	Oasis event_id	4545
coverage_id	int	4	Oasis coverage_id	150
sidx	int	4	Sample index	10
loss	float	4	The ground up loss value	5675.675

[Return to top](#)

fmtocsv

A component which converts the fmcalt output stream, or binary file with the same structure, to a csv file.

Standard input stream

Byte 1	Bytes 2-4	Description
2	1	loss

A binary file of the same format can be piped into fmtocsv.

Usage

```
$ [stdin component] | fmtocsv > [output].csv
$ fmtocsv < [stdin].bin > [output].csv
```

Example

```
$ eve 1 1 | getmodel | gulcalc -r -S100 -a1 -i - | fmcalt | fmtocsv > fmcalt.csv
$ fmtocsv < fmcalt.bin > fmcalt.csv
```

Output

Csv file with the following fields;

Name	Type	Bytes	Description	Example
event_id	int	4	Oasis event_id	4545
output_id	int	4	Oasis output_id	5
sidx	int	4	Sample index	10
loss	float	4	The insured loss value	5375.675

[Return to top](#)

summarycalctocsv

A component which converts the summarycalc output stream, or binary file with the same structure, to a csv file.

Standard input stream

Byte 1	Bytes 2-4	Description
3	1	summary

A binary file of the same format can be piped into summarycalctocsv.

Usage

```
$ [stdin component] | summarycalctocsv > [output].csv
$ summarycalctocsv < [stdin].bin > [output].csv
```

Example

```
$ eve 1 1 | getmodel | gulcalc -r -S100 -a1 -i - | fmcalt | summarycalc -f -1 - | summarycalctocsv
> summarycalc.csv
$ summarycalctocsv < summarycalc.bin > summarycalc.csv
```

Output

Csv file with the following fields;

Name	Type	Bytes	Description	Example
event_id	int	4	Oasis event_id	4545
summary_id	int	4	Oasis summary_id	3
sidx	int	4	Sample index	10
loss	float	4	The insured loss value	5375.675

[Return to top](#)

gultobin

A component which converts gulcalc data in csv format into gulcalc binary item stream (1/1).

Input file format

Name	Type	Bytes	Description	Example
event_id	int	4	Oasis event_id	4545
item_id	int	4	Oasis item_id	300
sidx	int	4	Sample index	10
loss	float	4	The ground up loss value	5675.675

- For each event_id and item_id combination, a record for sidx = -1 (mean) and sidx = -2 (standard deviation) must be included at the top, even if the loss values are zero.
- For each event_id and item_id combination, one or more sampled losses for sidx > 0 may be provided, but records for samples with zero loss may be omitted.

Parameters

-S, the number of samples must be provided. This can be equal to or greater than maximum sample index value that appears in the csv data. -t, the stream type of either 1 for the deprecated item stream or 2 for the loss stream. This is an optional parameter with default value 2.

Usage

```
$ gultobin [parameters] < [input].csv | [stdin component]
$ gultobin [parameters] < [input].csv > [output].bin
```

Example

```
$ gultobin -S100 < gulcalci.csv | fmcalt > fmcalt.bin
$ gultobin -S100 < gulcalci.csv > gulcalci.bin
$ gultobin -S100 -t1 < gulcalci.csv > gulcalci.bin
$ gultobin -S100 -t2 < gulcalci.csv > gulcalci.bin
```

Standard output stream

Byte 1	Bytes 2-4	Description
1	1	gulcalc item
2	1	gulcalc loss

[Return to top](#)

[Go to 4.6. Validation Components](#)

[Back to Contents](#)

4.6 Validation components

The following components run validity checks on csv format files:

Model data files

- **validatedamagebin** checks damage bin dictionary for validity.
- **validatefootprint** checks event footprint for validity.
- **validatevulnerability** checks vulnerability data for validity.
- **crossvalidation** performs validation checks across damage bin dictionary, event footprint and vulnerability data.

Oasis input files

- **validateoasisfiles** performs validation checks across coverages, items, fm policytc, fm programme and fm profile data.

Model data files

validatedamagebin

The following checks are performed on the damage bin dictionary:

- Each line contains 4 or 5 values.
- First bin index is 1.
- Bin indices are contiguous.
- Interpolation lies inside range.

In addition, warnings are issued in the following cases:

- Lower limit of first bin is not 0.
- Upper limit of last bin is not 1.
- Deprecated `interval_type` column included.
- Interpolation lies within range but not in the bin centre.

The checks can be performed on `damage_bin_dict.csv` from the command line:

```
$ validatedamagebin < damage_bin_dict.csv
```

The checks are also performed by default when converting damage bin dictionary files from csv to binary format:

```
$ damagebintobin < damage_bin_dict.csv > damage_bin_dict.bin
```

```
# Suppress validation checks with -N argument
```

```
$ damagebintobin -N < damage_bin_dict.csv > damage_bin_dict.bin
```

validatefootprint

The following checks are performed on the event footprint:

- Each line contains 4 values.
- Total probability for each event-areaperil combination is 1.
- Event IDs listed in ascending order.
- For each event ID, areaperils IDs listed in ascending order.
- No duplicate intensity bin IDs for each event-areaperil combination.

Should all checks pass, the maximum value of `intensity_bin_index` is given, which is a required input for `footprinttobin`.

The checks can be performed on `footprint.csv` from the command line:

```
$ validatefootprint < footprint.csv
```

The checks are also performed by default when converting footprint files from csv to binary format:

```
$ footprinttobin -i {number of intensity bins} < footprint.csv
```

```
# Suppress validation checks with -N argument
```

```
$ footprinttobin -i {number of intensity bins} -N < footprint.csv
```

validatevulnerability

The following checks are performed on the vulnerability data:

- Each line contains 4 values.
- Total probability for each vulnerability-intensity bin combination is 1.
- Vulnerability IDs listed in ascending order.
- For each vulnerability ID, all intensity bin IDs are present and listed in ascending order.
- For each vulnerability-intensity bin combination, damage bin IDs are contiguous and start at bin index 1.

Should all checks pass, the maximum value of `damage_bin_id` is given, which is a required input for `vulnerabilitytobin`.

The checks can be performed on `vulnerability.csv` from the command line:

```
$ validatevulnerability < vulnerability.csv
```

The checks are also performed by default when converting vulnerability files from csv to binary format:

```
$ vulnerabilitytobin -d {number of damage bins} < vulnerability.csv > vulnerability.bin
```

```
# Suppress validation checks with -N argument
```

```
$ vulnerabilitytobin -d {number of damage bins} -N < vulnerability.csv > vulnerability.bin
```

crossvalidation

The following checks are performed across the damage bin dictionary, event footprint and vulnerability data:

- Damage bin IDs in the vulnerability data are subset of those in the damage bin dictionary.
- Intensity bin IDs in the event footprint are subset of those in the vulnerability data.

The checks can be performed on `damage_bin_dict.csv`, `footprint.csv` and `vulnerability.csv` from the command line:

```
$ crossvalidation -d damage_bin_dict.csv -f footprint.csv -s vulnerability.csv
```

Input oasis files

validateoasisfiles

The following checks are performed across the coverages, items, fm policytc, fm programme and fm profile data:

- 1-to-1 relationship between `agg_id` in `fm_programme.csv` and `item_id` in `items.csv` when `level_id` = 1.
- `coverage_id` in `items.csv` matches those in `coverages.csv`.
- `policytc_id` in `fm_policytc.csv` matches those in `fm_profile.csv`.
- (`level_id`, `agg_id`) pairs in `fm_policytc.csv` are present as (`level_id`, `to_agg_id`) pairs in `fm_programme.csv`.
- When `level_id` = `n` > 1, `from_agg_id` corresponds to a `to_agg_id` from `level_id` = `n` - 1.

The checks can be performed on `coverages.csv`, `items.csv`, `fm_policytc.csv`, `fm_programme.csv` and `fm_profile.csv` from the command line, specifying the directory these files are located in:

```
$ validateoasisfiles -d path/to/output/directory
```

The Ground Up Losses (GUL) flag `g` can be specified to only perform checks on `items.csv` and `coverages.csv`:

```
$ validateoasisfiles -g -d /path/to/output/directory
```

[Return to top](#)

[Go to 5. Financial Module](#)

[Back to Contents](#)

5. Financial Module

The Oasis Financial Module is a data-driven process design for calculating the losses on (re)insurance contracts. It has an abstract design in order to cater for the many variations in contract structures and terms. The way Oasis works is to be fed data in order to execute calculations, so for the insurance calculations it needs to know the structure, parameters and calculation rules to be used. This data must be provided in the files used by the Oasis Financial Module:

- **fm_programme**: defines how coverages are grouped into accounts and programmes
- **fm_profile**: defines the layers and terms
- **fm_policytc**: defines the relationship of the contract layers
- **fm_xref**: specifies the summary level of the output losses

This section explains the design of the Financial Module which has been implemented in the **fmcalc** component.

- Runtime parameters and usage instructions for **fmcalc** are covered in [4.1 Core Components](#).
- The formats of the input files are covered in [4.3 Data Conversion Components](#).

In addition, there is a separate github repository [ktest](#) which is an extended test suite for ktools and contains a library of financial module worked examples provided by Oasis Members with a full set of input and output files.

Note that other reference tables are referred to below that do not appear explicitly in the kernel as they are not directly required for calculation. It is expected that a front end system will hold all of the exposure and policy data and generate the above four input files required for the kernel calculation.

Scope

The Financial Module outputs sample by sample losses by (re)insurance contract, or by item, which represents the individual coverage subject to economic loss from a particular peril. In the latter case, it is necessary to 'back-allocate' losses when they are calculated at a higher policy level. The Financial Module can output retained loss or ultimate net loss (UNL) perspectives as an option, and at any stage in the calculation.

The output contains anonymous keys representing the (re)insurance policy (**agg_id** and **layer_id**) at the chosen output level (**output_id**) and a loss value. Losses by sample number (**idx**) and event (**event_id**) are produced. To make sense of the output, this output must be cross-referenced with Oasis dictionaries which contain the meaningful business information.

The Financial Module does not support multi-currency calculations.

Profiles

Profiles are used throughout the Oasis framework and are meta-data definitions with their associated data types and rules. Profiles are used in the Financial Module to perform the elements of financial calculations used to calculate losses to (re)insurance policies. For anything other than the most simple policy which has a blanket deductible and limit, say, a profile do not represent a policy structure on its own, but rather is to be used as a building block which can be combined with other building blocks to model a particular financial contract. In this way it is possible to model an unlimited range of structures with a limited number of profiles.

The FM Profiles form an extensible library of calculations defined within the **fmcalc** code that can be invoked by specifying a particular **calcrule_id** and providing the required data values such as deductible and limit, as described below.

Supported Profiles

See Appendix [B FM Profiles](#) for more details.

Design

The Oasis Financial Module is a data-driven process design for calculating the

losses on insurance policies. It is an abstract design in order to cater for the many variations and has four basic concepts:

1. A **programme** which defines which **items** are grouped together at which levels for the purpose of providing loss amounts to policy terms and conditions. The programme has a user-definable profile and dictionary called **prog** which holds optional reference details such as a description and account identifier. The prog table is not required for the calculation and therefore does not appear in the kernel input files.
2. A policytc **profile** which provides the parameters of the policy's terms and conditions such as limit and deductible and calculation rules.
3. A **policytc** cross-reference file which associates a policy terms and conditions profile to each programme level aggregation.
4. A **xref** file which specifies how the output losses are summarized.

The profile not only provides the fields to be used in calculating losses (such as limit and deductible) but also which mathematical calculation (calcrule_id) to apply.

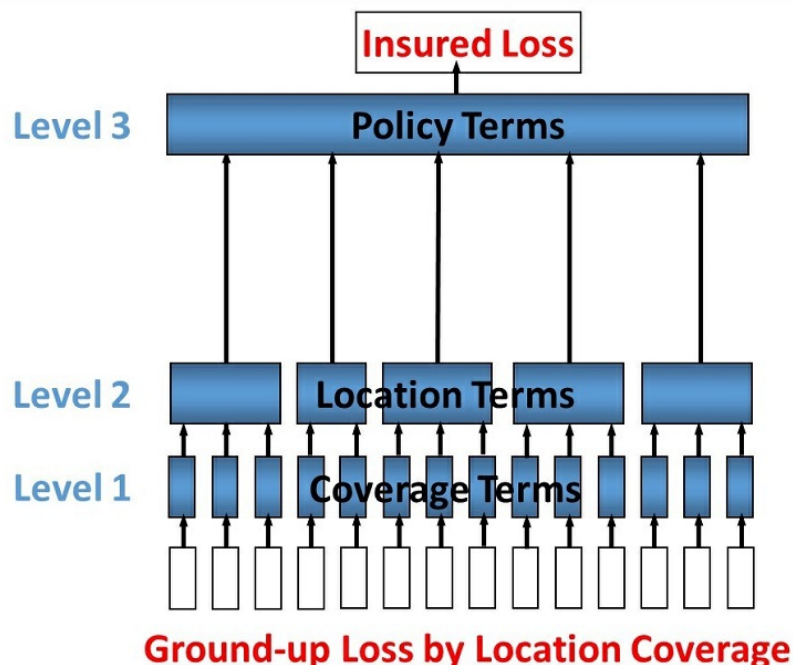
Data requirements

The Financial Module brings together three elements in order to undertake a calculation:

- Structural information, notably which items are covered by a set of policies.
- Loss values of items.
- Policy profiles and profile values.

There are many ways an insurance loss can be calculated with many different terms and conditions. For instance, there may be deductibles applied to each element of coverage (e.g. a buildings damage deductible), some site-specific deductibles or limits, and some overall policy deductibles and limits and share. To undertake the calculation in the correct order and using the correct items (and their values) the structure and sequence of calculations must be defined. This is done in the **programme** file which defines a hierarchy of groups across a number of **levels**. Levels drive the sequence of calculation. A financial calculation is performed at successive levels, depending on the structure of policy terms and conditions. For example there might be 3 levels representing coverage, site and policy terms and conditions.

Figure 1. Example 3-level programme hierarchy

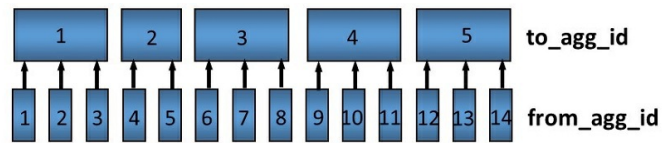


Groups are defined within levels and they represent aggregations of losses on which to perform the financial calculations. The grouping fields are called from_agg_id and to_agg_id which represent a grouping of losses at the previous level and the present level of the hierarchy, respectively.

Each level calculation applies to the to_agg_id groupings in the hierarchy. There is no calculation applied to the from_agg_id groupings at level 1 - these ids directly correspond to the ids in the loss input.

Figure 2. Example level 2 grouping

Level 2



Loss values

The initial input is the ground-up loss (GUL) table, generally coming from the main Oasis calculation of ground-up losses. Here is an example, for a two events and 1 sample (idx=1):

event_id	item_id	sidx	loss
1	1	1	100,000
1	2	1	10,000
1	3	1	2,500
1	4	1	400
2	1	1	90,000
2	2	1	15,000
2	3	1	3,000
2	4	1	500

The values represent a single ground-up loss sample for items belonging to an account. We use "programme" rather than "account" as it is more general characteristic of a client's exposure protection needs and allows a client to have multiple programmes active for a given period. The linkage between account and programme can be provided by a user defined **prog** dictionary, for example;

prog_id	account_id	prog_name
1	1	ABC Insurance Co. 2016 renewal

Items 1-4 represent Structure, Other Structure, Contents and Time Element coverage ground up losses for a single property, respectively, and this example is a simple residential policy with combined property coverage terms. For this policy type, the Structure, Other Structure and Contents losses are aggregated, and a deductible and limit is applied to the total. A separate set of terms, again a simple deductible and limit, is applied to the "Time Element" coverage which, for residential policies, generally means costs for temporary accommodation. The total insured loss is the sum of the output from the combined property terms and the time element terms.

Programme

The actual items falling into the programme are specified in the **programme** table together with the aggregation groupings that go into a given level calculation:

from_agg_id	level_id	to_agg_id
1	1	1
2	1	1
3	1	1
4	1	2
1	2	1
2	2	1

Note that from_agg_id for level_id=1 is equal to the item_id in the input loss table (but in theory from_agg_id could represent a higher level of grouping, if required).

In level 1, items 1, 2 and 3 all have to_agg_id=1 so losses will be summed together before applying the combined deductible and limit, but item 4 (time element) will be treated separately (not aggregated) as it has to_agg_id = 2. For level 2 we have all 4 items losses (now represented by two groups from_agg_id =1 and 2 from the previous level) aggregated together as they have the same to_agg_id = 1.

Profile

Next we have the profile description table, which list the profiles representing general policy types. Our example is represented by two general profiles which specify the input fields and mathematical operations to perform. In this example, the profile for the combined coverages and time is the same (albeit with different values) and requires a limit, a deductible, and an associated calculation rule, whereas the profile for the policy requires a limit, attachment, and share, and an associated calculation rule.

Profile description	calcrule_id
deductible and limit	1
deductible and/or attachment, limit and share	2

There is a "profile value" table for each profile containing the applicable policy terms, each identified by a policytc_id. The table below shows the list of policy terms for calcrule_id 1.

policytc_id	deductible1	limit1
1	1,000	1,000,000
2	2,000	18,000

And next, for calcrule_id 2, the values for the overall policy attachment, limit and share

policytc_id	deductible1	attachment1	limit1	share1
3	0	1,000	1,000,000	0.1

In practice, all profile values are stored in a single flattened format which contains all supported profile fields (see fm_profile in [4.3 Data Conversion Components](#)), but conceptually they belong in separate profile value tables.

The flattened file is;

fm_profile

policytc_id	calcrule_id	deductible1	deductible2	deductible3	attachme
1	1	1,000	0	0	0
1	1	2,000	0	0	0
1	2	0	0	0	1,000

For any given profile we have one standard rule **calcrule_id**, being the mathematical function used to calculate the losses from the given profile's fields. More information about the functions can be found in [FM Profiles](#).

Policytc

The **policytc** table specifies the (re)insurance contracts (this is a combination of agg_id and layer_id) and the separate terms and conditions which will be applied to each layer_id/agg_id for a given level. In our example, we have a limit and deductible with the same value applicable to the combination of the first three items, a limit and deductible for the fourth item (time) in level 1, and then a limit, attachment, and share applicable at level 2 covering all items. We'd represent this in terms of the distinct agg_ids as follows:

layer_id	level_id	agg_id	policytc_id
1	1	1	1
1	1	2	2
1	2	1	3

In words, the data in the table mean;

At Level 1;

Apply policytc_id (terms and conditions) 1 to (the sum of losses represented by) agg_id 1

Apply policytc_id 2 to agg_id 2

Then at level 2;

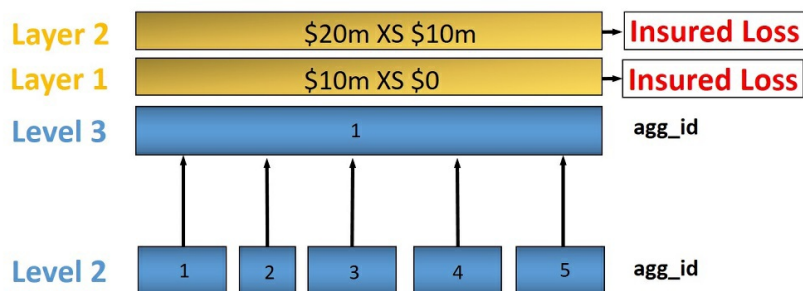
Apply policytc_id 3 to agg_id 1

Levels are processed in ascending order and the calculated losses from a previous level are summed according to the groupings defined in the programme table which become the input losses to the next level.

Layers

Layers can be used to model multiple sets of terms and conditions applied to the same losses, such as excess policies. For the lower level calculations and in the general case where there is a single contract, layer_id should be set to 1. For a given level_id and agg_id, multiple layers can be defined by setting layer_id =1,2,3 etc, and assigning a different calculation policytc_id to each.

Figure 3. Example of multiple layers



For this example at level 3, the polycytc data might look as follows;

layer_id	level_id	agg_id	polycytc_id
1	3	1	22
2	3	1	23

Output and back-allocation

Losses are output by event, output_id and sample. The table looks like this;

event_id	output_id	sidx	loss
1	1	1	455.24
2	1	1	345.6

The output_id is specified by the user in the xref table, and is a unique combination of agg_id and layer_id. For instance;

output_id	agg_id	layer_id
1	1	1
2	1	2

The output_id must be specified consistently with the back-allocation rule. Losses can either output at the contract level or back-allocated to the lowest level, which is item_id, using one of three command line options. There are three meaningful values here – don't allocate (0) used typically for all levels where a breakdown of losses is not required in output, allocate back to items (1) in proportion to the input (ground up) losses, or allocate back to items (2) in proportion to the losses from the prior level calculation.

\$ fmcalt -a0 # Losses are output at the contract level and not back-allocated
 \$ fmcalt -a1 # Losses are back-allocated to items on the basis of the input losses (e.g. ground up loss)
 \$ fmcalt -a2 # Losses are back-allocated to items on the basis of the prior level losses

The rules for specifying the output_ids in the xref table are as follows;

- Rule 0: there is an output_id for every agg_id and layer_id of the final level in the polycytc table
- Rule 1: there is an output_id for every from_agg_id of the first level in the programme table, and for every layer_id in the final level of the polycytc table
- Rule 2: there is an output_id for every from_agg_id of the first level in the programme table, and for every layer_id in the final level of the polycytc table

To make sense of this, if there is more than one output at the contract level, then each one must be back-allocated to all of the items, with each individual loss represented by a unique output_id.

To avoid unnecessary computation, it is recommended not to back-allocate unless losses are required to be reported at a more detailed level than the contract level (site or zip code, for example). In this case, losses are re-aggregated up from item level (represented by output_id in fmcalt output) in summarycalc, using the fmsummaryxref table.

Reinsurance

The first run of fmcalt is designed to calculate the primary or direct insurance losses from the ground up losses of an exposure portfolio. fmcalt has been designed to be recursive, so that the 'gross' losses from the first run can be streamed back in to second and subsequent runs of fmcalt, each time with a different set of input files representing reinsurance contracts, and can output either the reinsurance gross loss, or net loss. There are two modes of output;

- **gross** meaning the loss to the policies or reinsurance contracts, and
- **net** being the difference between the input loss and the policy/contract losses

net loss is output when the command line parameter -n is used, otherwise output loss is gross by default.

Supported reinsurance types

The types of reinsurance supported by the Financial Module are;

- **Facultative** proportional or excess
- **Quota share** with event limit
- **Surplus share** with event limit
- **Per risk** with event limit
- **Catastrophe excess of loss** occurrence only

Required files

Second and subsequent runs of fmcalc require the same four fm files fm_programme, fm_policytc, fm_profile, and fm_xref.

This time, the hierarchy specified in fm_programme must be consistent with the range of output_ids from the incoming stream of losses, as specified in the fm_xref file from the previous iteration. Specifically, this means the range of values in from_agg_id at level 1 must match the range of values in output_id.

For example;

fm_xref (iteration 1)

output_id	agg_id	layer_id
1	1	1
2	1	2

fm_programme (iteration 2)

from_agg_id	level_id	to_agg_id
1	1	1
2	1	2
1	2	1
2	2	1

The abstraction of from_agg_id at level 1 from item_id means that losses needn't be back-allocated to item_id after every iteration of fmcalc. In fact, performance will be improved when back-allocation is minimised.

Example - Quota share reinsurance

Using the two layer example from above, here's an example of the fm files for a simple quota share treaty with 50% ceded and 90% placed covering both policy layers.

The command to run the direct insurance followed by reinsurance might look like this;

```
$ fmcalc -p direct < guls.bin | fmcalc -p ri1 -n > ri1_net.bin
```

In this command, ground up losses are being streamed into fmcalc to calculate the insurance losses, which are streamed into fmcalc again to calculate the reinsurance net loss. The direct insurance fm files would be located in the folder 'direct' and the reinsurance fm files in the folder 'ri1'. The -n flag in the second call of fmcalc results in net losses being output to the file 'ri1_net.bin'. These are the losses to the insurer net of recoveries from the quota share treaty.

The fm_xref file from the direct insurance (first) iteration is

fm_xref

output_id	agg_id	layer_id
1	1	1
2	1	2

The fm files for the reinsurance (second) iteration would be as follows;

fm_programme

from_agg_id	level_id	to_agg_id
1	1	1
2	1	1

fm_policytc

layer_id	level_id	agg_id	policytc_id
1	1	1	1

fm_profile

policytc_id	calcrule_id	deductible1	deductible2	deductible3	attachme
1	25	0	0	0	0



fm_xref

output_id	agg_id	layer_id
1	1	1

Inuring priority

The Financial Module can support unlimited inuring priority levels for reinsurance. Each set of contracts with equal inuring priority would be calculated in one iteration. The net losses from the first inuring priority are streamed into the second inuring priority calculation, and so on.

Where there are multiple contracts with equal inuring priority, these are implemented as layers with a single iteration.

The net calculation for iterations with multiple layers is;

net loss = max(0, input loss - layer1 loss - layer2 loss - ... - layer n loss)

[Return to top](#)

[Go to 6. Workflows](#)

[Back to Contents](#)

6. Workflows

ktools is capable of multiple output workflows. This brings much greater flexibility, but also more complexity for users of the toolkit.

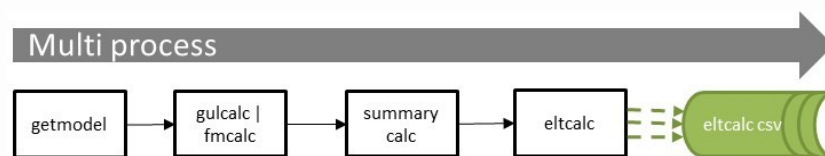
This section presents some example workflows, starting with single output workflows and then moving onto more complex multi-output workflows. There are some python scripts provided which execute some of the illustrative workflows using the example data in the repository. It is assumed that workflows will generally be run across multiple processes, with the number of processes being specified by the user.

1. Portfolio summary level insured loss event loss table

In this example, the core workflow is run through `fmcalc` into `summarycalc` and then the losses are summarized by summary set 2, which is "portfolio" summary level. This produces multiple output files when run with multiple processes, each containing a subset of the event set. The output files can be concatenated together at the end.

```
eve 1 2 | getmodel | gulcalc -r -S100 -i - | fmcalc | summarycalc -f -2 - | eltcalc > elt_p1.csv
eve 2 2 | getmodel | gulcalc -r -S100 -i - | fmcalc | summarycalc -f -2 - | eltcalc > elt_p2.csv
```

Figure 1. eltcalc workflow



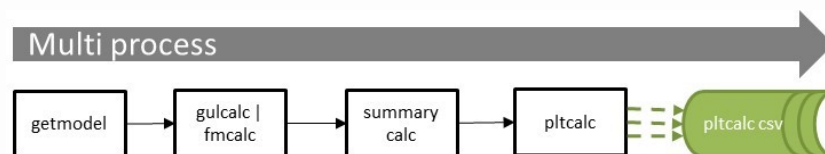
See example script `eltcalc_example.py` ***

2. Portfolio summary level insured loss period loss table

This is very similar to the first example, except the summary samples are run through `pltcalc` instead. The output files can be concatenated together at the end.

```
eve 1 2 | getmodel | gulcalc -r -S100 -i - | fmcalc | summarycalc -f -2 - | pltcalc > plt_p1.csv
eve 2 2 | getmodel | gulcalc -r -S100 -i - | fmcalc | summarycalc -f -2 - | pltcalc > plt_p2.csv
```

Figure 2. pltcalc workflow



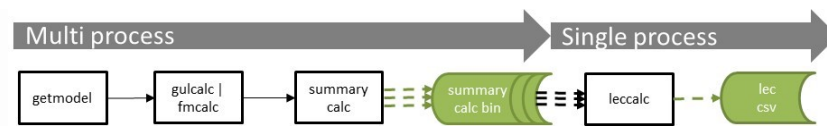
See example script `pltcalc_example.py` ***

3. Portfolio summary level full uncertainty aggregate and occurrence loss exceedance curves

In this example, the summary samples are calculated as in the first two examples, but the results are output to the work folder. Until this stage the calculation is run over multiple processes. Then, in a single process, `leccalc` reads the `summarycalc` binaries from the work folder and computes two loss exceedance curves in a single process. Note that you can output all eight loss exceedance curve variants in a single `leccalc` command.

```
eve 1 2 | getmodel | gulcalc -r -S100 -i - | fmcalc | summarycalc -f -2 - > work/summary2/p1.bin
eve 2 2 | getmodel | gulcalc -r -S100 -i - | fmcalc | summarycalc -f -2 - > work/summary2/p1.bin
leccalc -Ksummary2 -F lec_full_uncertainty_agg.csv -f lec_full_uncertainty_occ.csv
```

Figure 3. leccalc workflow



See example script [leccalc_example.py](#) ***

4. Portfolio summary level average annual loss

Similarly to lec curves, the samples are run through to summarycalc, and the summarycalc binaries are output to the work folder. Until this stage the calculation is run over multiple processes. Then, in a single process, aalcalc reads the summarycalc binaries from the work folder and computes the aal output.

```
eve 1 2 | getmodel | gulcalc -r -S100 -i - | fmcalt | summarycalc -f -2 work/summary2/p1.bin
eve 2 2 | getmodel | gulcalc -r -S100 -i - | fmcalt | summarycalc -f -2 work/summary2/p2.bin
aalcalc -Ksummary2 > aal.csv
```

Figure 4. aalcalc workflow



See example script [aalcalc_example.py](#) ***

Multiple output workflows

5. Ground up and insured loss workflows

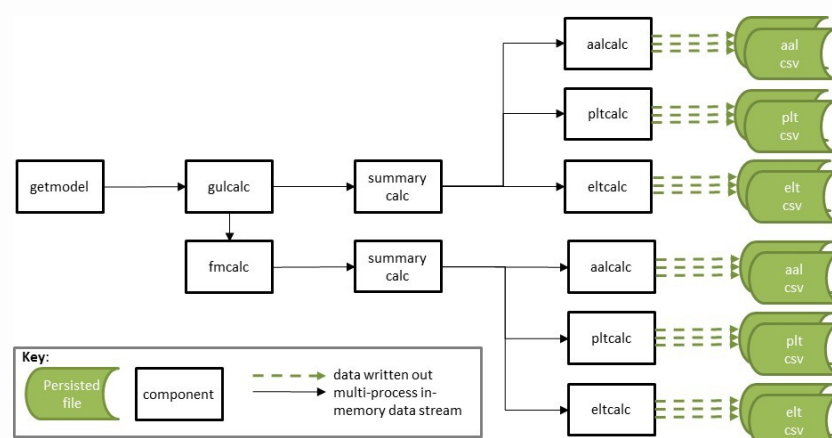
gulcalc can generate two output streams at once: item level samples to pipe into fmcalt, and coverage level samples to pipe into summarycalc. This means that outputs for both ground up loss and insured loss can be generated in one workflow. This is done by writing one stream to a file or named pipe, while streaming the other to standard output down the pipeline.

```
eve 1 2 | getmodel | gulcalc -r -S100 -i gulcalci1.bin -c - | summarycalc -g -2 - | eltcalt >
gul_elt_p1.csv
eve 2 2 | getmodel | gulcalc -r -S100 -i gulcalci2.bin -c - | summarycalc -g -2 - | eltcalt >
gul_elt_p2.csv
fmcalt < gulcalci1.bin | summarycalc -f -2 - | eltcalt > fm_elt_p1.csv
fmcalt < gulcalci2.bin | summarycalc -f -2 - | eltcalt > fm_elt_p2.csv
```

Note that the gulcalc item stream does not need to be written off to disk, as it can be sent to a 'named pipe', which keeps the data in-memory and kicks off a new process. This is easy to do in Linux (but harder in Windows).

Figure 5 illustrates an example workflow.

Figure 5. Ground up and insured loss example workflow



See example script [gulandfm_example.py](#) ***

6. Multiple summary level workflows

Summarycalc is capable of summarizing samples to up to 10 different user-defined levels for ground up loss and insured loss. This means that different outputs can be run on different summary levels. In this example, event loss tables for two different

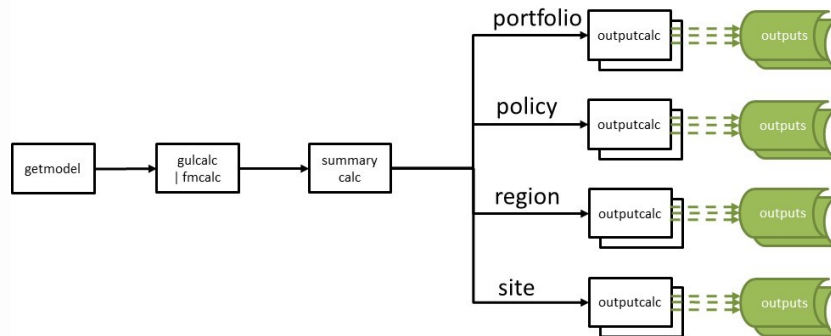
summary levels are generated.

```
eve 1 2 | getmodel | gulcalc -r -S100 -i - | fmcalt | summarycalc -f -1 s1/p1.bin -2 s2/p1.bin
eve 2 2 | getmodel | gulcalc -r -S100 -i - | fmcalt | summarycalc -f -1 s1/p2.bin -2 s2/p2.bin
elcalc < s1/p1.bin > elt_s1_p1.csv
elcalc < s1/p2.bin > elt_s1_p2.csv
elcalc < s2/p1.bin > elt_s2_p1.csv
elcalc < s2/p2.bin > elt_s2_p2.csv
```

Again, the summarycalc streams can be sent to named pipes rather than written off to disk.

Figure 6 illustrates multiple summary level streams, each of which can go to different output calculations.

Figure 6. Multiple summary level workflows

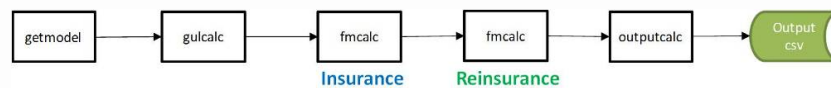


Financial Module workflows

The fmcalt component can be used recursively in order to apply multiple sets of policy terms and conditions, in order to support reinsurance. Figure 7 shows a simple example workflow of a direct insurance calculation followed by a reinsurance calculation.

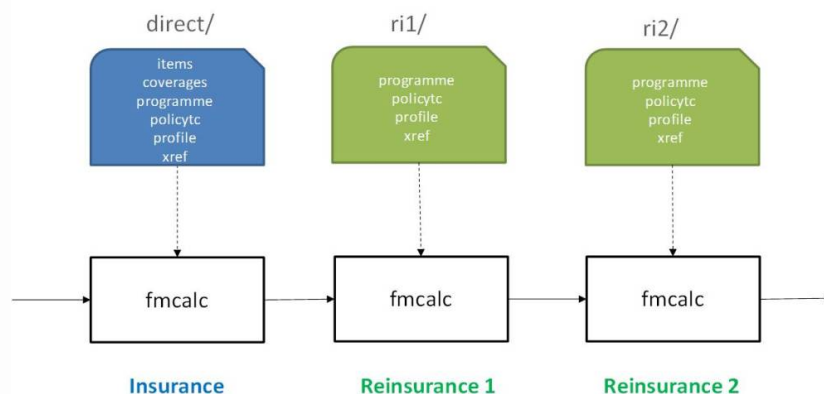
```
eve 1 2 | getmodel | gulcalc -r -S100 -i - | fmcalt -p direct | fmcalt -p ri1 -n > fmcalt_1.bin
eve 2 2 | getmodel | gulcalc -r -S100 -i - | fmcalt -p direct | fmcalt -p ri1 -n > fmcalt_2.bin
```

Figure 7. Multiple fmcalt workflow



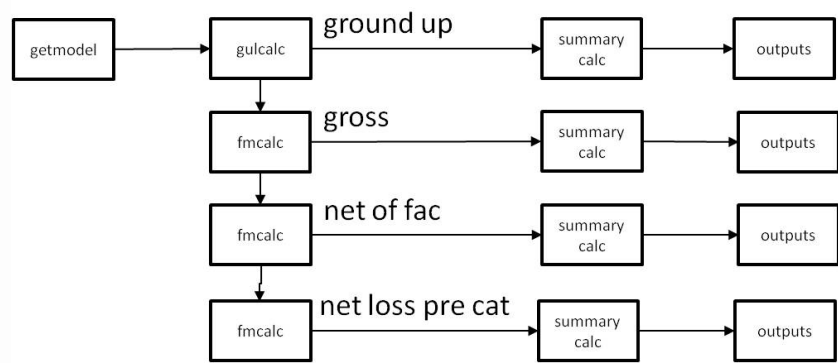
Each call of fmcalt requires the same input files, so it is necessary to specify the location of the files for each call using the command line parameter -p and the relative folder path. Figure 8 demonstrates the required files for three consecutive calls of fmcalt.

Figure 8. Multiple fmcalt workflow



It is possible to generate all of the outputs for each call of fmcalt in the same workflow, enabling multiple financial perspective reports, as shown in Figure 9.

Figure 9. Multiple fmcalt outputs workflow



[Return to top](#)

[Go to Appendix A Random numbers](#)

[Back to Contents](#)

Appendix A: Random numbers

Simple uniform random numbers are assigned to each event, group and sample number to sample ground up loss in the gulcalc process. A group is a collection of items which share the same group_id, and is the method of supporting spatial correlation in ground up loss sampling in Oasis and ktools.

Correlation

Items (typically representing, in insurance terms, the underlying risk coverages) that are assigned the same group_id will use the same random number to sample damage for a given event and sample number. Items with different group_ids will be assigned independent random numbers. Therefore sampled damage is fully correlated within groups and fully independent between groups, where group is an abstract collection of items defined by the user.

The item_id, group_id data is provided by the user in the items input file (items.bin).

Methodology

The method of assigning random numbers in gulcalc uses a random number index (ridx), an integer which is used as a position reference into a list of random numbers. S random numbers corresponding to the runtime number of samples are drawn from the list starting at the ridx position.

There are three options in ktools for choosing random numbers to apply in the sampling process.

1. Generate dynamically during the calculation

Usage

Use -R{number of random numbers} as a parameter. Optionally you may use -s{seed} to make the random numbers repeatable.

Example

```
$ gulcalc -S00 -R1000000 -i -
```

This will run 100 samples drawing from 1 million dynamically generated random numbers. They are simple uniform random numbers.

```
$ gulcalc -S00 -s123 -R1000000 -i -
```

This will run 100 samples drawing from 1 million seeded random numbers (repeatable)

Method

Random numbers are sampled dynamically using the Mersenne twister pseudo random number generator (the default RNG of the C++ v11 compiler). A sparse array capable of holding R random numbers is allocated to each event. The ridx is generated from the group_id and number of samples S using the following modulus function;

$$\text{ridx} = \text{mod}(\text{group_id} \times P1, R)$$

- P1 is the first prime number which is bigger than the number of samples, S.

This formula pseudo-randomly assigns ridx indexes to each group_id between 0 and 999,999.

As a ridx is sampled, the section in the array starting at the ridx position of length S is populated with random numbers unless they have already been populated, in which case the existing random numbers are re-used.

The array is cleared for the next event and a new set of random numbers is generated.

2. Use numbers from random number file

Usage

Use -r as a parameter

Example

```
$ gulcalc -S100 -r -i -
```

This will run 100 samples using random numbers from file random.bin in the static sub-directory.

Method

The random number file(s) is read into memory at the start of the gulcalc process.

The ridx is generated from the sample index (sidx), event_id and group_id using the following modulus function;

$$\text{ridx} = \text{sidx} + \text{mod}(\text{group_id} \times P1 \times P3 + \text{event_id} \times P2, R)$$

- R is the divisor of the modulus, equal to the total number of random numbers in the list.
- P1 and P2 are the first two prime numbers which are greater than half of R.
- P3 is the first prime number which is bigger than the number of samples, S.

This formula pseudo-randomly assigns a starting position index to each event_id and group_id combo between 0 and R-1, and then S random numbers are drawn by incrementing the starting position by the sidx.

3. Generate automatically seeded random numbers (no buffer)

Usage

Default option

Example

```
$ gulcalc -S100 -i -
```

This option will produce repeatable random numbers seeded from a combination of the event_id and group_id. The difference between this option and method 1 with the fixed seed is that there is no limit on the number of random numbers generated, and you do not need to make a decision on the buffer size. This will impact performance for large analyses.

Method

For each event_id and group_id, the seed is calculated as follows;

```
s1 = mod(group_id * 1543270363, 2147483648);  
s2 = mod(event_id * 1943272559, 2147483648); seed = mod(s1 + s2 ,  
2147483648)
```

[Return to top](#)

[Go to Appendix B FM Profiles](#)

[Back to Contents](#)

Appendix B: FM Profiles

This section specifies the attributes and rules for the following list of Financial module profiles.

Profile description	calcrule_id
Do nothing (pass losses through)	100
deductible and limit	1
deductible with attachment, limit and share	2
franchise deductible and limit	3
deductible % TIV and limit	4
deductible and limit % loss	5
deductible % TIV	6
deductible, minimum and maximum deductible, with limit	7
deductible and minimum deductible, with limit	8
limit with deductible % limit	9
deductible and maximum deductible	10
deductible and minimum deductible	11
deductible	12
deductible, minimum and maximum deductible	13
limit only	14
deductible and limit % loss	15
deductible % loss	16
deductible % loss with attachment, limit and share	17
deductible % tiv with attachment, limit and share	18
deductible % loss with min and/or max deductible	19
reverse franchise deductible	20
deductible % tiv with min and max deductible	21
reinsurance % ceded, limit and % placed	22
reinsurance limit and % placed	23
reinsurance excess terms	24
reinsurance proportional terms	25
deductible % loss with min and/or max deductible and limit	26
% tiv trigger and % tiv step payout with limit	27
% tiv trigger and % loss step payout	28
% tiv trigger and % tiv step payout	29
% tiv trigger and % limit step payout	30
% tiv trigger and monetary amount step payout	31
monetary amount trigger and % loss step payout with limit	32
deductible % loss with limit	33
deductible with attachment and share	34
deductible % loss with min and/or max deductible and limit % loss	35
deductible with min and/or max deductible and limit % loss	36
% tiv trigger and % loss step payout with limit	37
conditional coverage payouts based on prior step payouts	38

calcrule_id	d1	d2	d3	a1	l1	sh1	sh2	sh3	st	ts	te	ps	pe	l2
100														
1	x				x									
2	x			x	x	x								
3	x				x									
4	x				x									
5	x				x									
6	x													
7	x	x	x		x									
8	x	x			x									
9	x				x									
10	x		x											
11	x	x												
12	x													
13	x	x	x											
14					x									
15	x				x									
16	x													
17	x			x	x	x								
18	x			x	x	x								
19	x	x	x											
20	x													
21	x	x	x											
22					x	x	x	x						
23					x		x	x						
24				x	x	x	x	x						
25						x	x	x						
26	x	x	x		x									
27	x				x					x	x	x	x	x
28	x									x	x	x	x	x
29	x									x	x	x	x	x
30	x				x					x	x	x	x	x
31	x									x	x	x	x	x
32					x					x	x		x	x
33	x				x									
34	x			x		x								
35	x	x	x		x									
36	x	x	x		x									
37	x				x					x	x	x	x	x
38										x	x	x		x

The fields with an x are those which are required by the profile. The full names of the fields are as follows;

Short name	Profile field name
d1	deductible_1
d2	deductible_2
d3	deductible_3
a1	attachment_1
l1	limit_1
sh1	share_1
sh2	share_2
sh3	share_3
st	step_id
ts	trigger_start
te	trigger_end
ps	payout_start
pe	payout_end
l2	limit_2
sc1	scale_1
sc2	scale_2

An allocation rule can be assigned to each call of fncalc, which determines whether calculated losses should be back-allocated to the contributing items, and if so how. This is specified via the command line parameter -a.

The choices are as follows;

Allocrule description	allocrule_id
Don't back-allocate losses (default if no parameter supplied)	0
Back allocate losses to items in proportion to input loss	1
Back-allocate losses to items in proportion to prior level loss	2
Back-allocate losses to items in proportion to prior level loss (reinsurance)	3

Effective deductibles

Often there are more than one hierarchical levels with deductibles, and there is a choice of methods of accumulation of deductibles through the hierarchy. Whenever a rule with a deductible is used in the loss calculation then it is accumulated through the calculation in an **effective_deductible** variable. The effective deductible is the smaller of the deductible amount and the loss.

All deductibles amounts calculated from the deductible_1 field are simply additive through the hierarchy.

At any level, the user can specify a calcrule using a minimum and/or maximum deductible which changes the way that effective deductibles are accumulated.

For a minimum deductible specified in calcrules using the deductible_2 field, the calculation increases the effective_deductible carried forward from the previous levels up to the minimum deductible if it is smaller.

For a maximum deductible specified in calcrules using the deductible_3 field, the calculation decreases the effective_deductible carried forward from the previous levels down to the maximum deductible if it is larger.

Adjustment of loss for prior level limits

Loss adjustments due to minimum and maximum deductibles may lead to breaching or falling short of prior level limits. For instance, an increase in loss due a policy maximum deductible being applied can lead to a breach of site limit that applied at the prior calculation level. Conversely, a decrease in loss due to policy minimum deductible can leave the loss falling short of a site limit applied at the prior calculation level. In these situations the prior level limits are carried through and reapplied in all calcrules that have minimum and/or maximum deductibles.

We introduce the following variables;

- **over limit** is the sum of the amount by which limits are exceeded at the prior level calculation (positive measure)
- **under limit** is the smaller of the effective deductible and sum of the amount by which losses fall short of limits at the prior level calculation (positive measure). It represents the amount by which the loss can be adjusted upwards due to the enforcement of a maximum deductible at the current level.
- **loss delta** is the amount by which the loss would change due to the enforcement of a minimum or maximum ignoring prior level limits (positive for loss increase and negative for loss decrease). The loss is adjusted by an amount up to the loss delta depending on the values carried in the under and over limit variables

Scenarios for under and over limit

The over and under limit variables are initialised when there exist prior level limits. The possible cases are;

Case	Under limit	Over limit	Meaning
1	0	0	All prior level losses are exactly at their limits
2	0	>0	Some prior level losses are over limit and none are under limit
3	>0	0	Some prior level losses are under limit and none are over limit
4	>0	>0	Some prior level losses are over limit and some are under limit

When the loss delta is positive;

- in cases 1 and 2 the loss is not adjusted upwards because the prior level losses are already at their limits. The loss delta is added to the carried over limit.
- in cases 3 and 4 the loss is adjusted upwards to a maximum of the under limit. Losses are therefore adjusted up to the prior level limits and any excess loss

delta is added to the over limit variable. The loss increase is subtracted from the carried under limit.

When the loss delta is negative;

- in cases 1 and 2 the loss delta is added to the over limit until the over limit is zero and any excess loss delta is used to adjust the loss downwards. Losses that are over limit by more than the loss delta remain at their limits, and decrease only if the over limit is fully eroded by the loss delta.
- in cases 3 and 4 losses are adjusted down fully by the loss delta, or to zero if the loss delta is greater than the initial loss. The loss adjustment is added to the carried under limit.

Current calculation level limits may also apply and these are used to update the over limit and under limit measures to carry through to the next level.

Calcrules

In the following notation;

- x.loss is the input loss to the calculation
- x.effective_deductible is the input effective deductible to the calculation (where required)
- loss is the calculated loss

1. Deductible and limit

Attributes	Example
policytc_id	1
calcrule_id	1
deductible_1	50000
limit_1	900000

Calculation logic

```
loss = x.loss - deductible_1;  
if (loss < 0) loss = 0;  
if (loss > limit_1) loss = limit_1;
```

2. Deductible, attachment, limit and share

Attributes	Example
policytc_id	1
calcrule_id	2
deductible_1	70000
attachment_1	0
limit_1	1000000
share_1	0.1

Calculation logic

```
loss = x.loss - deductible_1  
if (loss < 0) loss = 0;  
if (loss > (attachment_1 + limit_1)) loss = limit_1;  
else loss = loss - attachment_1;  
if (loss < 0) loss = 0;  
loss = loss * share_1;
```

3. Franchise deductible and limit

Attributes	Example
policytc_id	1
calcrule_id	3
deductible_1	100000
limit_1	1000000

Calculation logic

```
if (x.loss < deductible_1) loss = 0;  
else loss = x.loss;  
if (loss > limit_1) loss = limit_1;
```

5. Deductible and limit as a proportion of loss

Attributes	Example
policytc_id	1
calcrule_id	5
deductible_1	0.05
limit_1	0.3

Calculation logic

```
loss = x.loss - (x.loss * deductible_1);  
if (loss > (x.loss * limit_1)) loss = x.loss * lim;
```

9. Limit with deductible as a proportion of limit

Attributes	Example
policytc_id	1
calcrule_id	9
deductible_1	0.05
limit_1	100000

Calculation logic

```
loss = x.loss - (deductible_1 * limit_1);  
if (loss < 0) loss = 0;  
if (loss > limit_1) loss = limit_1;
```

10. Maximum deductible

If the effective deductible carried forward from the previous level exceeds the maximum deductible, the effective deductible is decreased to the maximum deductible value

Attributes	Example
policytc_id	1
calcrule_id	10
deductible_3	40000

Calculation logic

```
if (x.effective_deductible > deductible_3) {  
    loss = x.loss + x.effective_deductible - deductible_3;  
    if (loss < 0) loss = 0;  
}  
else {  
    loss = x.loss;  
}
```

11. Minimum deductible

If the effective deductible carried forward from the previous level is less than the minimum deductible, the deductible is increased to the total loss or the minimum deductible value, whichever is greater.

Attributes	Example
policytc_id	1
calcrule_id	11
deductible_2	70000

Calculation logic

```
if (x.effective_deductible < deductible_2) {  
    loss = x.loss + x.effective_deductible - deductible_2;  
    if (loss < 0) loss = 0;  
}  
else {  
    loss = x.loss;  
}
```

12. Deductible only

Attributes	Example
policytc_id	1
calcrule_id	12
deductible_1	100000

Calculation logic

```
loss = x.loss - deductible_1;
if (loss < 0) loss = 0;
```

14. Limit only

Attributes	Example
policytc_id	1
calcrule_id	14
limit	100000

Calculation logic

```
loss = x.loss;
if (loss > limit_1) loss = limit_1;
```

15. Limit as a proportion of loss

Attributes	Example
policytc_id	1
calcrule_id	15
limit_1	0.3

Calculation logic

```
loss = x.loss * limit_1;
```

16. Deductible as a proportion of loss

Attributes	Example
policytc_id	1
calcrule_id	16
deductible_1	0.05

Calculation logic

```
loss = x.loss - (x.loss * deductible_1);
if (loss < 0) loss = 0;
```

[Return to top](#)

[Go to Appendix C Multi-peril model support](#)

[Back to Contents](#)

Appendix C: Multi-peril model support

ktools now supports multi-peril models through the introduction of the `coverage_id` in the data structures.

Ground up losses apply at the “Item” level in the Kernel which corresponds to “interest coverage” in business terms, which is the element of financial loss that can be associated with a particular asset. In ktools, `item_id` represents the element of financial loss and `coverage_id` represents the asset with its associated total insured value. If there is more than one item per coverage (as defined in the items data) then each item represents an element of financial loss from a particular peril contributing to the total loss for the asset. For each item, the identification of the peril is held in the `areaperil_id`, which is a unique key representing a combination of the location (area) and peril.

Multi-peril damage calculation

Ground up losses are calculated by multiplying the damage ratio for an item by the total insured value of its associated coverage (defined in the coverages data). The questions are then; how are these losses combined across items, and how are they correlated?

There are a few ways in which losses can be combined and the first example in ktools uses a simple rule, which is to sum the losses for each coverage and cap the overall loss to the total insured value. This is what you get when you use the `-c` parameter in `gulcalc` to output losses by 'coverage'.

In v3.1.0 the method of combining losses became function-driven using the `gulcalc` command line parameter `-a` as a few standard approaches have emerged. These are;

Allocation option	Description
0	Do nothing (suitable for single sub-peril models with one item per coverage)
1	Sum damage ratios and cap to 1. Back-allocate in proportion to contributing subperil loss
2	Total damage = maximum subperil damage. Back-allocate all to the maximum contributing subperil loss
3	Multiplicative method for combining damage. Back-allocate in proportion to contributing subperil loss

Allocation options 0,1,2 have been implemented to date.

Correlation of item damage is generic in ktools, as damage can either be 100% correlated or independent (see [Appendix A Random Numbers](#)). This is no different in the multi-peril case when items represent different elements of financial loss to the same asset, rather than different assets. More sophisticated methods of multi-peril correlation have been implemented for particular models, but as yet no standard approach has been implemented in ktools.

Note that ground up losses by item can be passed into the financial module unallocated (allocation method 0) using the `gulcalc` option `-i`, or allocated using the `gulcalc` option `-a1 -i`. If the item ground up losses are passed though unallocated then the limit of total insured value must be applied as part of the financial module calculations, to prevent the ground up loss exceeding the coverage TIV.

[Return to top](#)

[Back to Contents](#)